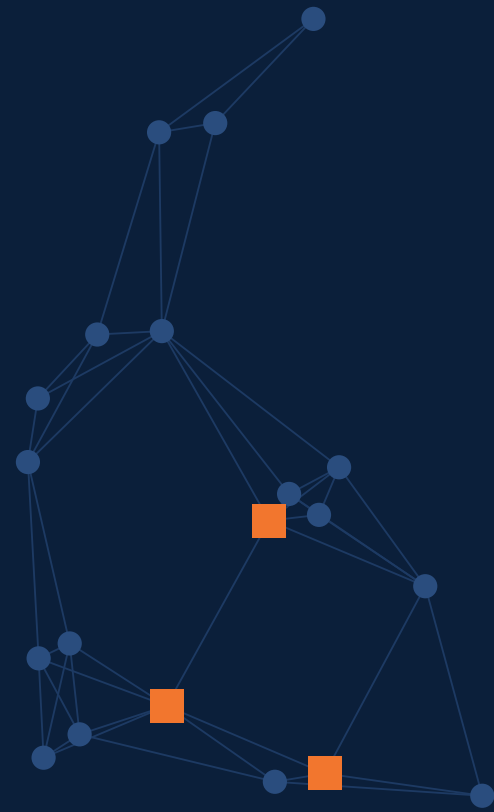


PROJET 5 — DOSSIER COMPLET

Recherche Opérationnelle × Supply Chain Planning

Optimiser une supply chain de bout en bout par MILP



Localisation d'entrepôts, dimensionnement des lots de production et tournées de livraison : trois niveaux de décision, un seul fil conducteur — résolu avec Python, PuLP et le solveur CBC.

MÉTHODE MILP (PuLP + CBC) · programmation dynamique · heuristiques

PÉRIMÈTRE 21 usines clientes réelles en Allemagne · 9 entrepôts candidats

LIVRABLE Réseau recommandé + plan de production + tournées + code

NIVEAU Projet étudiant Génie Industriel — qualité professionnelle

3

niveaux de décision
reliés entre eux

21

usines clientes
(géographie réelle)

9,10

M€/an : coût total
du réseau recommandé

–52 %

baisse max. du coût de
livraison (tournées)

SOMMAIRE

Table des matières

0 Résumé exécutif	3
1 Cadrage du projet	3
2 Les bases du MILP à maîtriser	6
3 Niveau stratégique — localisation d'entrepôts	9
4 Niveau tactique — dimensionnement des lots	14
5 Niveau opérationnel — tournées de livraison	19
6 Boucler les niveaux de décision	23
7 Synthèse type A3	26
8 Reproduire et s'approprier le projet	26
9 Présenter ce projet en entretien	28
10 Limites & prolongements	30
A Annexe — code source complet	31
B Annexe — glossaire	52

COMMENT LIRE CE GUIDE

Ce document présente un projet **réellement exécuté** : chaque chiffre, tableau et graphique provient du code fourni en annexe (Python 3, PuLP 3.3, solveur CBC). Il est construit pour être lu comme un cours, puis rejoué et modifié par vous-même.

Parcours conseillé : lisez d'abord le résumé (section 0) et le cours condensé (section 2) ; suivez ensuite les trois niveaux dans l'ordre (sections 3 à 5), car chacun réutilise les notions du précédent ; la section 6 est la plus précieuse pour l'entretien ; terminez par les exercices de la section 8, sans regarder les indices au premier essai.

0 Résumé exécutif

Le projet en une page

Contexte. Une entreprise tunisienne de composants automobiles (cas fictif « TunAuto Components », Ben Arous) livre **21 usines clientes en Allemagne** — VW, BMW, Mercedes, Audi, Ford, Opel et des équipementiers Tier-1 (ZF, Bosch, Schaeffler), soit 81 900 palettes par an. Les marchandises traversent la Méditerranée en roulier (RoRo) jusqu'au port de Gênes, puis remontent par camion. Trois questions se posent, à trois horizons différents : **où installer des entrepôts** en Allemagne, **comment dimensionner les lots de production** à l'usine, et **comment organiser les tournées** de livraison quotidiennes.

Méthode. Chaque question est formulée comme un programme linéaire en nombres entiers (MILP) et résolue avec PuLP et CBC, puis comparée systématiquement à des méthodes plus simples (heuristiques, programmation dynamique) pour mesurer ce que l'optimisation apporte réellement. Enfin, les niveaux sont **bouclés** : le coût des tournées calculé au niveau opérationnel sert à recalculer l'hypothèse du niveau stratégique.

0,10 s

pour trouver le réseau optimal
(198 variables)

+78 %

de surcharge si l'on planifie les
produits séparément

−40 %

coût de livraison moyen grâce
aux tournées

×2,3

écart entre coût aval supposé
et coût réel des tournées

Résultats. (1) Avec l'hypothèse initiale, le réseau optimal compte trois entrepôts — Kassel, Nürnberg, Ulm — pour 6,74 M€ par an ; une méthode « centre de gravité » (K-means) coûte de 2,8 % à 10,4 % de plus. (2) À l'usine, la planification produit par produit (Wagner-Whitin) surcharge la ligne goulot jusqu'à 1 118 h pour 630 h disponibles ; le modèle CLSP multi-produits donne un plan faisable à 153,6 k€ avec 98 heures supplémentaires ciblées. (3) Les tournées optimisées réduisent le coût de livraison de 30 à 52 % par rapport à des envois directs. (4) Surtout : le coût réel des tournées (0,317 €/palette·km) est 2,3 fois supérieur à l'hypothèse initiale. Après recalage, le réseau recommandé devient **Hannover · Erfurt · Nuernberg · Mannheim** pour 9,10 M€ par an, et le modèle stratégique prédit désormais correctement le coût réel.

CE QUI REND CE PROJET DIFFÉRENT DE VOS PROJETS 1 À 4

Ce projet montre trois choses qu'un recruteur Supply Chain Planning cherche : savoir **formuler** un problème métier en MILP, savoir **prouver** qu'une solution est bonne (optimum, borne, gap), et savoir **remettre en cause ses propres hypothèses** quand les niveaux de décision se contredisent.

1 Cadrage du projet

Phase 0 — cadrage & business case

1.1 — Pourquoi ce projet dans votre portfolio

Votre portfolio couvre déjà la maintenance prédictive (machine learning), la simulation Lean (SimPy), l'ordonnancement (programmation par contraintes, CP-SAT) et la prévision de production (PFA Tunistools). Il lui manque deux briques très demandées dans les offres de PFE allemandes « Supply Chain Planning », « Logistikplanung » ou « Operations Research » : la **programmation linéaire en nombres entiers classique** et la **logistique de distribution**. Les outils industriels de planification (SAP IBP, Kinaxis, o9, les modules d'optimisation des TMS) reposent en grande partie sur des modèles

MILP : savoir en écrire un, le résoudre et en critiquer le résultat est une compétence directement transférable.

Projet	Outil principal	Ce qu'il prouve	Ce que le projet 5 ajoute
1 — Maintenance prédictive	scikit-learn	Machine learning, déséquilibre de classes	Décision prescriptive, pas seulement prédictive
2 — Cellule en U	SimPy	Simulation, Lean, dimensionnement	Optimisation exacte (pas d'essais-erreurs)
4 — Job-shop	OR-Tools CP-SAT	Ordonnancement, exact vs heuristique	MILP classique, formulations fortes/faibles
PFA Tunistools	pandas, Ridge	Données réelles, prévision	Transformer une prévision en plan

1.2 — Le cas d'étude

L'entreprise fabrique trois références de composants usinés à Ben Arous et les exporte vers l'industrie automobile allemande. Le choix de ce cas n'est pas neutre : la Tunisie est l'un des premiers fournisseurs nord-africains de l'automobile européenne, et la question « comment servir l'Allemagne depuis la Tunisie ? » est un sujet réel que vous pouvez défendre en entretien avec conviction. Les flux sont les suivants :

- **Amont** : l'usine expédie par roulier Radès → Gênes, puis par camions complets (33 palettes) jusqu'aux entrepôts allemands. Le transport amont par la route coûte environ 0,052 €/palette·km.
- **Entrepôts** : 9 hubs logistiques candidats (Hamburg, Hannover, Dortmund, Kassel, Frankfurt, Erfurt, Nürnberg, Mannheim, Ulm), chacun avec un coût fixe annuel, une capacité et un coût de manutention.
- **Aval** : chaque usine cliente est livrée en juste-à-temps depuis un entrepôt, deux fois par jour.

HONNÊTÉTÉ MÉTHODOLOGIQUE

Géographie réelle, économie construite. Les coordonnées GPS des 21 usines et des 9 hubs sont réelles ; les distances routières sont estimées par la formule de haversine multipliée par un facteur de détour de 1,25 (valeur standard en Europe). Les volumes, coûts fixes et coûts unitaires sont des **ordres de grandeur construits**, pas des données d'entreprise. C'est exactement ce qu'il faut dire en entretien : la méthode est réelle et transférable, les chiffres seraient à recalculer avec des données terrain.

1.3 — Trois niveaux de décision, un seul fil conducteur

La planification d'une supply chain est classiquement découpée en trois niveaux hiérarchiques (Anthony, 1965 ; repris dans toute la littérature de *Supply Chain Management*). Chaque niveau fixe le cadre du niveau inférieur : on ne planifie pas des tournées tant qu'on ne sait pas d'où partent les camions.



Fig. 1 — La hiérarchie de planification suivie dans ce projet. Chaque niveau devient un modèle d'optimisation distinct.

1.4 — Objectifs (formulation SMART)

- **Spécifique** : choisir les entrepôts à ouvrir, planifier la production sur 12 mois et construire les tournées quotidiennes, en minimisant le coût total.
- **Mesurable** : coût annuel (€), taux d'utilisation des entrepôts, charge de la ligne goulot (h), kilomètres et nombre de camions par vague, écart à l'optimum (gap).
- **Atteignable** : outils gratuits (PuLP + CBC) ; taille des modèles compatible avec un ordinateur portable.
- **Réaliste** : géographie réelle, contraintes métier réelles (capacités, temps de réglage, 33 palettes par semi-remorque, temps de conduite).
- **Temporellement défini** : 3 à 4 semaines de travail personnel, découpées en 6 phases.

1.5 — Les données en un coup d'œil

Entrepôt candidat	Coût fixe / an	Capacité (pal/an)	Manutention / pal	Distance depuis Gênes (km)
Hamburg	520 k€	45 000	6,5 €	1 274
Hannover	430 k€	40 000	5,8 €	1 109
Dortmund	440 k€	40 000	5,8 €	996
Kassel	380 k€	35 000	5,2 €	961
Frankfurt	560 k€	45 000	6,8 €	793
Erfurt	350 k€	35 000	4,6 €	934
Nuernberg	450 k€	40 000	5,6 €	730
Mannheim	470 k€	40 000	5,9 €	707
Ulm	400 k€	35 000	5,4 €	564

Usine cliente	Palettes/an	Usine cliente	Palettes/an
Wolfsburg (VW)	7 800	Emden (VW)	3 200
Sindelfingen (Mercedes)	7 200	Baunatal (VW)	3 100

Usine cliente	Palettes/an	Usine cliente	Palettes/an
Ingolstadt (Audi)	6 900	Ruesselsheim (Opel)	2 800
Dingolfing (BMW)	6 100	Salzgitter (VW)	2 700
Muenchen (BMW)	5 200	Schweinfurt (ZF)	2 600
Bremen (Mercedes)	4 600	Friedrichshafen (ZF)	2 400
Leipzig (BMW/Porsche)	4 400	Herzogenaurach (Schaeffler)	2 300
Koeln (Ford)	3 900	Saarlouis (Ford)	2 100
Regensburg (BMW)	3 800	Bamberg (Bosch)	2 000
Neckarsulm (Audi)	3 600	Eisenach (Opel)	1 900
Zwickau (VW)	3 300		

Tableau 1 — Les 21 usines clientes et leur demande annuelle (total 81 900 palettes).

Paramètre	Valeur	Justification
Facteur de détour route / vol d'oiseau	1,25	Valeur standard pour le réseau routier européen
Coût d'un semi-remorque complet	1,70 €/km	Ordre de grandeur FTL en Allemagne (carburant, péage, chauffeur)
Transport amont	0,052 €/pal·km	1,70 € / 33 palettes : camions pleins
Transport aval (hypothèse initiale)	0,14 €/pal·km	Livraisons JIT, camions partiellement remplis — à vérifier (section 6)
Jours ouvrés / an	230	Calendrier des usines allemandes
Coût fixe par camion et par vague	180 €	Chauffeur + amortissement du tracteur

Tableau 2 — Paramètres de transport. L'hypothèse aval sera remise en cause en section 6.

2 Les bases du MILP à maîtriser

Cours condensé — à lire avant de coder

Cette section rassemble ce qu'il faut comprendre pour ne pas utiliser un solveur « en boîte noire ». Elle est volontairement courte : chaque notion est ensuite appliquée concrètement dans les sections 3 à 5.

2.1 — Du programme linéaire au MILP

Un **programme linéaire (LP)** minimise une fonction linéaire de variables continues sous des contraintes linéaires. Il se résout très vite, même avec des millions de variables (algorithme du simplexe, points intérieurs). Un **programme linéaire en nombres entiers mixte (MILP)** ajoute des variables entières — le plus souvent **binaires** (0 ou 1) — qui permettent de modéliser des **décisions** : ouvrir ou non un entrepôt, lancer ou non une production, emprunter ou non une route. Cette capacité a un prix : le problème devient NP-difficile en général.

	LP	MILP	CP-SAT (projet 4)
Variables	continues	continues + entières/binaires	entières, intervalles
Force	très rapide, dualité, prix marginaux	coûts fixes, logique oui/non, bornes prouvées	ordonnancement, contraintes logiques riches
Faiblesse	ne sait pas dire « ouvrir ou pas »	sensible à la qualité de la formulation	moins adapté aux gros flux continus

	LP	MILP	CP-SAT (projet 4)
Solveurs	HiGHS, CLP, Gurobi	CBC, HiGHS, Gurobi, CPLEX	OR-Tools

2.2 — Anatomie d'un modèle

Tout modèle d'optimisation, quel que soit le problème, s'écrit avec quatre ingrédients :

1. **Les ensembles et les données** : clients I , entrepôts J , demandes d_j , coûts c_{ij} ... Ce sont des nombres connus.
2. **Les variables de décision** : ce que le solveur choisit. Exemple : $y_j = 1$ si l'entrepôt j est ouvert.
3. **La fonction objectif** : ce que l'on minimise (coût) ou maximise (marge, service).
4. **Les contraintes** : ce qui doit être respecté (demande servie, capacité, précédence...).

Le réflexe à acquérir : **écrire le modèle mathématique sur papier avant d'écrire le code**. Le code PuLP en est ensuite une traduction presque ligne à ligne.

2.3 — Comment le solveur trouve (et prouve) l'optimum

CBC, comme tous les solveurs MILP, utilise le **branch-and-bound** (séparation et évaluation) enrichi de coupes :

1. Il résout d'abord la **relaxation LP** : le même problème où les variables binaires peuvent prendre n'importe quelle valeur entre 0 et 1. Sa valeur est une **borne inférieure** : l'optimum entier ne peut pas être meilleur.
2. Si une variable binaire vaut par exemple 0,45, il **sépare** : une branche avec cette variable à 0, une autre à 1, et recommence dans chaque branche.
3. Chaque solution entière trouvée donne une **borne supérieure**. Toute branche dont la relaxation est déjà pire que la meilleure solution connue est **élaguée**.
4. Le **gap** = (meilleure solution – meilleure borne) / meilleure solution. À 0 %, l'optimalité est **prouvée**. Si le temps s'arrête avant, on sait seulement que l'optimum se trouve dans l'intervalle.

À RETENIR

La vitesse d'un MILP dépend moins du nombre de variables que de la **qualité de la relaxation LP**. Deux formulations mathématiquement équivalentes (mêmes solutions entières) peuvent avoir des relaxations très différentes. On parle de formulation **forte** (relaxation proche de l'optimum entier) ou **faible**. Vous le mesurerez vous-même en section 3.6.

2.4 — Le « big-M » : un outil indispensable et dangereux

Pour dire « pas de production sans lancement », on écrit $x_t \leq M \cdot y_t$: si $y_t = 0$, la production est forcée à 0 ; si $y_t = 1$, elle est limitée par M . Le piège : choisir $M = 10^9$ « pour être sûr ». La relaxation LP peut alors mettre $y_t = 10^{-7}$ et payer une fraction infime du coût de lancement : la borne devient inutile et le solveur rame. **Règle : toujours prendre le plus petit M valide** (ici, la demande restante ou la capacité).

2.5 — PuLP en 15 lignes

Exemple minimal, exécuté tel quel : deux lignes de production, chacune avec un coût fixe d'ouverture, une capacité et un coût variable ; il faut produire 900 unités au moindre coût.

```

■ exemple_minimal.py
import pulp
m = pulp.LpProblem("deux_lignes", pulp.LpMinimize)
q = pulp.LpVariable.dicts("q", ["A", "B"], lowBound=0)           # quantites (continues)
y = pulp.LpVariable.dicts("ouvrir", ["A", "B"], cat="Binary")   # decisions (0/1)
cap, fixe, var = {"A": 600, "B": 800}, {"A": 2000, "B": 3500}, {"A": 3, "B": 2}

m += pulp.lpSum(fixe[l] * y[l] + var[l] * q[l] for l in "AB")    # objectif
m += q["A"] + q["B"] == 900                                     # demande
for l in "AB":
    m += q[l] <= cap[l] * y[l]                                   # capacite si ouverte

m.solve(pulp.PULP_CBC_CMD(msg=False))
print(pulp.LpStatus[m.status], pulp.value(m.objective))
# -> Optimal 7400.0 (A ouverte : 100 unites, B ouverte : 800 unites)

```

Aucune ligne seule ne suffit (600 et 800 < 900) : le solveur ouvre les deux et charge d'abord la ligne B, dont le coût variable est le plus faible. Toute la suite du projet utilise exactement cette grammaire : `LpVariable`, `lpSum`, `m += ...` pour l'objectif puis les contraintes.

2.6 — CBC, HiGHS, Gurobi : changer de solveur en une ligne

PuLP est un **langage de modélisation** : le même modèle peut être envoyé à différents solveurs. CBC est installé avec PuLP, gratuit et open source. HiGHS est un autre solveur open source moderne. Gurobi est commercial, généralement nettement plus rapide sur les gros MILP ; il propose des licences académiques gratuites pour les étudiants (à demander sur le site de Gurobi avec une adresse universitaire), et le paquet `gurobipy` installé par `pip` fonctionne sans licence pour de petits modèles (taille limitée).

```

■ changer_de_solveur.py
m.solve(pulp.PULP_CBC_CMD(msg=False, timeLimit=60)) # CBC (par défaut, gratuit)
m.solve(pulp.HiGHS(msg=False, timeLimit=60))        # HiGHS (pip install highspy)
m.solve(pulp.GUROBI(msg=False, timeLimit=60))       # Gurobi (pip install gurobipy + licence)

```

PIÈGE N°1 — LE STATUT « OPTIMAL » TROMPEUR

PuLP peut afficher « Optimal » alors que l'optimum n'est pas prouvé. Quand CBC s'arrête sur une limite de temps avec une solution entière, `pulp.LpStatus[m.status]` renvoie encore « Optimal » — vérifié dans ce projet. Le vrai diagnostic est dans `m.sol_status` : **1** = optimum prouvé, **2** = solution faisable non prouvée. Le code du niveau 3 lit ce champ et récupère la borne inférieure dans le journal de CBC pour calculer le gap. Ne présentez jamais une solution comme « optimale » sans avoir fait cette vérification.

PIÈGES N°2 À 5

Autres erreurs classiques : (1) unités incohérentes (€/palette mélangés avec €/camion) ; (2) oublier qu'une variable est continue par défaut (`cat="Binary"` ou `pulp.LpInteger`) ; (3) utiliser `sum()` au lieu de `pulp.lpSum()` sur de gros modèles (beaucoup plus lent) ; (4) lire `pulp.value(...)` sans vérifier qu'une solution existe (renvoie `None`).

3 Niveau stratégique — localisation d'entrepôts

3.1 — Le problème

Faut-il un grand entrepôt central ou plusieurs entrepôts régionaux ? Plus on en ouvre, plus les coûts fixes montent, mais plus les clients sont proches et moins le transport aval coûte. Le **problème de localisation avec capacités** (*Capacitated Facility Location Problem*, CFLP) tranche cet arbitrage de façon optimale. Chaque client est livré par un seul entrepôt (*single sourcing*) : une usine automobile veut un point de livraison unique, pas deux transporteurs qui se croisent au quai.

3.2 — Formulation mathématique

$$\begin{aligned} \min \quad & \sum_{j \in J} f_j y_j + \sum_{i \in I} \sum_{j \in J} d_i c_{ij} x_{ij} && \text{coûts fixes + coûts variables} \\ & \sum_{j \in J} x_{ij} = 1 \quad \forall i \in I && (1) \text{ chaque client est servi une fois} \\ & \sum_{i \in I} d_i x_{ij} \leq K_j y_j \quad \forall j \in J && (2) \text{ capacité, seulement si ouvert} \\ & x_{ij} \leq y_j \quad \forall i \in I, j \in J && (3) \text{ liaison « forte »} \\ & y_j \in \{0, 1\}, \quad x_{ij} \in \{0, 1\} && \text{décisions binaires} \\ & c_{ij} = \alpha \text{dist}(\text{Genes}, j) + h_j + \beta \text{dist}(j, i) && \text{coût par palette : amont + manutention + aval} \end{aligned}$$

f_j est le coût fixe annuel de l'entrepôt j , K_j sa capacité, h_j son coût de manutention, d_i la demande du client i , $\alpha = 0,052$ et $\beta = 0,14$ €/palette·km. La contrainte (3) est **redondante** pour les solutions entières (la contrainte (2) interdit déjà de servir depuis un entrepôt fermé), mais elle change radicalement la relaxation LP — c'est l'objet de la section 3.6.

3.3 — Le code clé

```
■ level1_cflp.py – extrait
m = pulp.LpProblem("CFLP", pulp.LpMinimize)
y = pulp.LpVariable.dicts("open", J, 0, 1, pulp.LpBinary)
x = pulp.LpVariable.dicts("assign", (I, J), 0, 1, pulp.LpBinary) # single sourcing

m += (pulp.lpSum(f[j] * y[j] for j in J)
      + pulp.lpSum(d[i] * c[i, j] * x[i][j] for i in I for j in J))

for i in I:
    m += pulp.lpSum(x[i][j] for j in J) == 1 # (1) demande servie a 100 %
for j in J:
    m += pulp.lpSum(d[i] * x[i][j] for i in I) <= cap[j] * y[j] # (2) capacite
for i in I:
    for j in J:
        m += x[i][j] <= y[j] # (3) liaison forte

m.solve(pulp.PULP_CBC_CMD(msg=False, timeLimit=120))
```

3.4 — Résultat : le réseau optimal (hypothèse initiale)

Le modèle compte 198 variables et 219 contraintes. CBC trouve et **prouve** l'optimum en 0,10 seconde : ouvrir **Kassel, Nürnberg et Ulm**, pour un coût annuel de 6,74 M€. La distance moyenne pondérée entre un client et son entrepôt est de 168 km.

Réseau optimal — hypothèse initiale (0,14 €/pal·km aval)

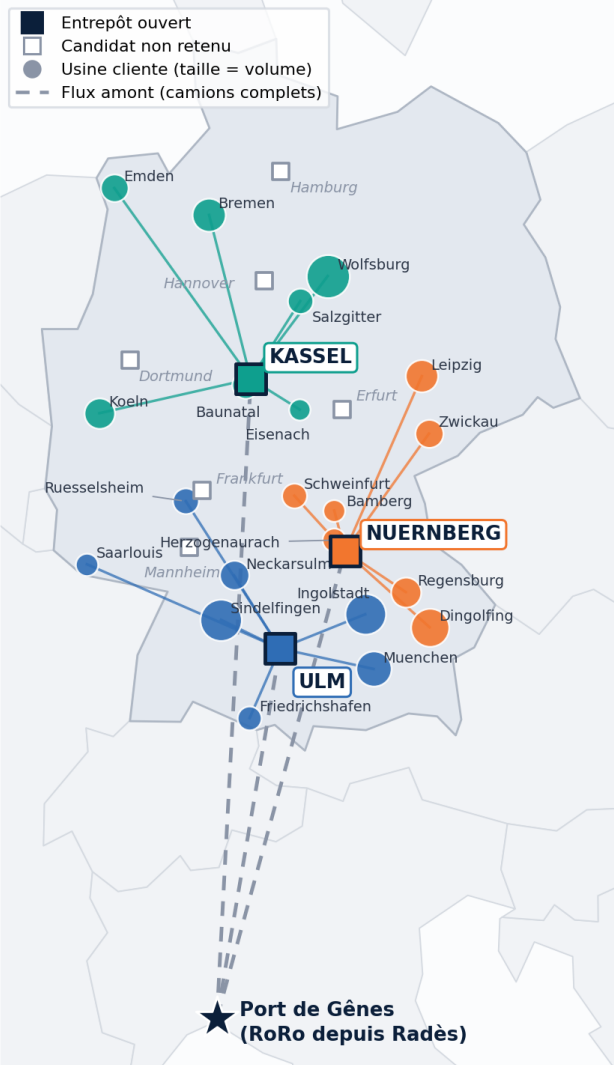


Fig. 2 — Réseau optimal avec l'hypothèse initiale. Chaque couleur représente la zone d'un entrepôt ; la taille des points est proportionnelle au volume.

Entrepôt ouvert	Clients servis	Volume (pal/an)	Capacité	Utilisation
Kassel	7	27 200	35 000	78 %
Nuernberg	7	24 500	40 000	61 %
Ulm	7	30 200	35 000	86 %

1,23 M€
coûts fixes

3,14 M€
transport amont

0,44 M€
manutention

1,93 M€
transport aval

Le transport amont depuis Gênes représente 47 % du coût total : c'est lui qui « tire » le réseau vers le sud. Remarquez que Köln, Saarlouis et Rüsselsheim, à l'ouest, n'ont pas d'entrepôt proche : ouvrir Mannheim ou Dortmund coûterait plus en coût fixe que ce qu'il ferait gagner en transport aval.

3.5 — Combien d'entrepôts ? La courbe en U

En ajoutant la contrainte $\sum_j y_j = p$ et en faisant varier p , on obtient la courbe classique de l'arbitrage coûts fixes / transport — l'équivalent logistique de la courbe de dimensionnement d'effectif de votre projet 2.

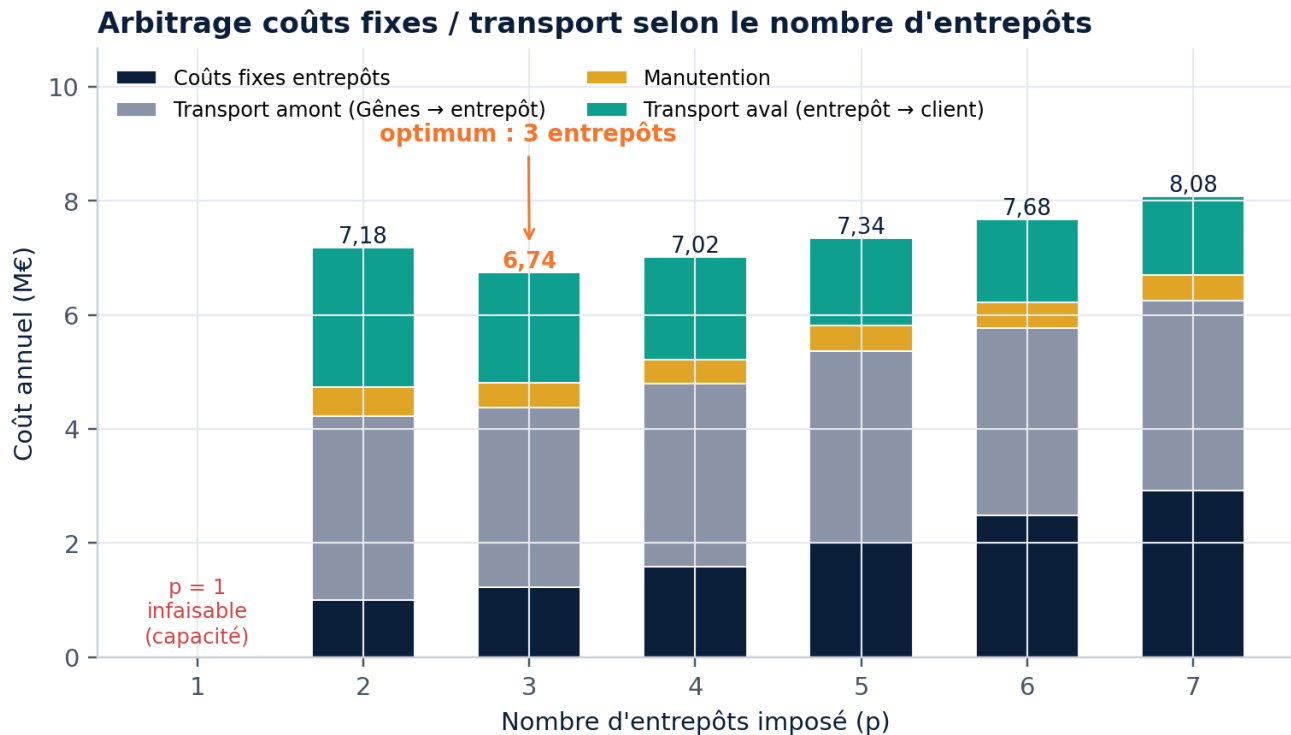


Fig. 3 — Coût annuel optimal pour chaque nombre d'entrepôts imposé. Avec un seul entrepôt, aucune capacité ne suffit : le modèle est infaisable.

Passer de 2 à 3 entrepôts fait gagner 439 k€ par an ; passer de 3 à 4 en coûte 274 k€. Au-delà de 3, chaque entrepôt supplémentaire ajoute plus de coûts fixes qu'il n'économise de kilomètres.

3.6 — Mono-sourcing, formulation forte ou faible : deux expériences

Expérience 1 — Combien coûte l'exigence « un seul entrepôt par client » ?

En relâchant l'intégrité des x_{ij} (un client peut être livré par plusieurs entrepôts), l'optimum passe de 6 742 112 € à 6 741 069 € : l'écart n'est que de **1 043 €** (0,015 %). Seul Dingolfing serait partagé entre Nürnberg et Ulm. Conclusion métier : l'exigence de mono-sourcing, souvent imposée par les clients, ne coûte presque rien ici — un argument chiffré à apporter en réunion.

Expérience 2 — La relaxation LP selon la formulation

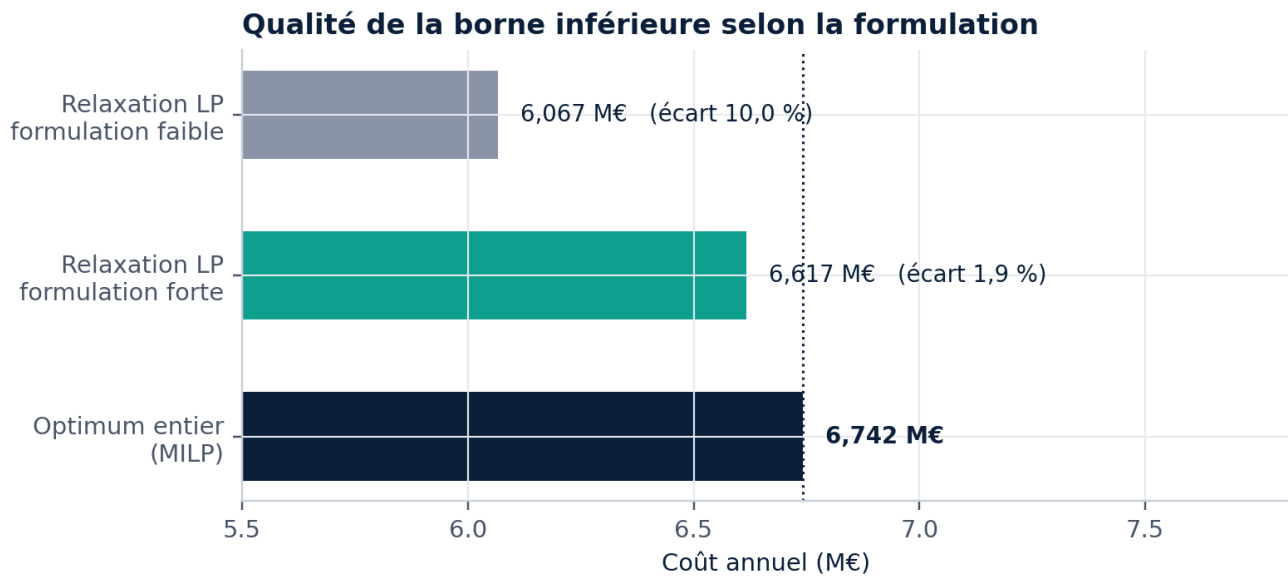


Fig. 4 — Borne inférieure fournie par la relaxation LP, avec et sans la contrainte de liaison (3). Même problème, même optimum entier.

Sans la contrainte (3), la relaxation LP « ouvre » sept entrepôts à des fractions comme 0,09 ou 0,38 et sous-estime le coût de 10,0 %. Avec (3), l'écart tombe à 1,9 % : Kassel et Ulm sont déjà à 1 dans la relaxation, seul Nürnberg vaut 0,45. Sur ce petit modèle, les deux versions se résolvent vite (0,56 s pour la faible, 0,10 s pour la forte), mais sur un réseau de 200 sites et 2 000 clients, cette différence de borne se traduit par des heures de calcul.

À RETENIR

Une contrainte « redondante » n'est pas inutile : elle peut **resserrer la relaxation**. Savoir expliquer pourquoi $x_{ij} \leq y_j$ améliore la borne est une question d'entretien classique en Recherche Opérationnelle.

3.7 — La décision est-elle robuste ?

Les coûts fixes et le coût du transport aval sont les hypothèses les plus incertaines. On relance le modèle 25 fois en les multipliant par des facteurs de 0,5 à 2,5 :

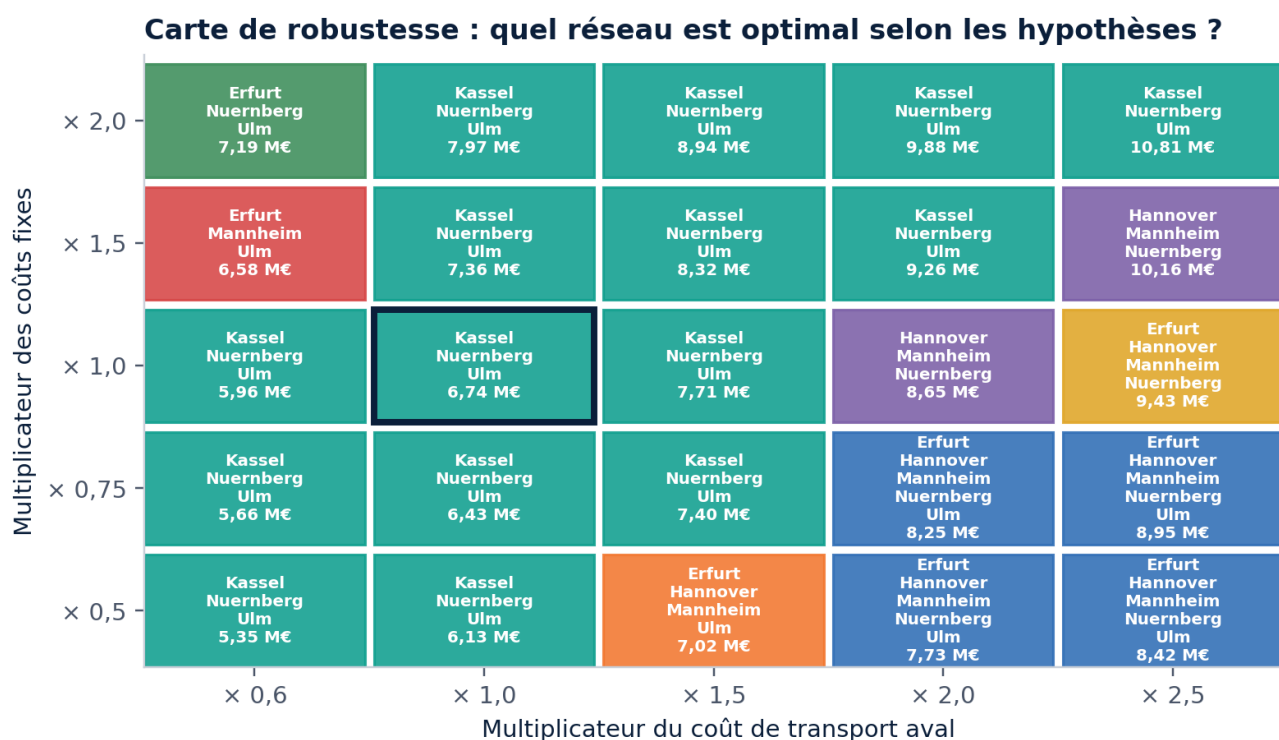


Fig. 5 — Réseau optimal pour chaque combinaison d'hypothèses. La case encadrée correspond aux hypothèses initiales.

Kassel · Nürnberg · Ulm reste optimal dans la majorité des cases proches des hypothèses initiales : toute la colonne « transport aval ×1 » (coûts fixes de ×0,5 à ×2) et presque toute la colonne ×1,5. En revanche, dès que le transport aval double, le réseau bascule vers 3 ou 4 entrepôts plus proches des clients (Hannover, Mannheim, Erfurt...). Gardez cette carte en tête : la section 6 montrera que c'est exactement ce qui se passe.

3.8 — Et sans optimisation ? La méthode du centre de gravité

Une approche intuitive, enseignée en cours de logistique et qui rejoint votre projet de Data Mining, consiste à regrouper les clients en 3 zones par **K-means pondéré** par les volumes, puis à ouvrir l'entrepôt candidat le plus proche de chaque centre de gravité. On l'a lancée avec 20 initialisations aléatoires différentes :

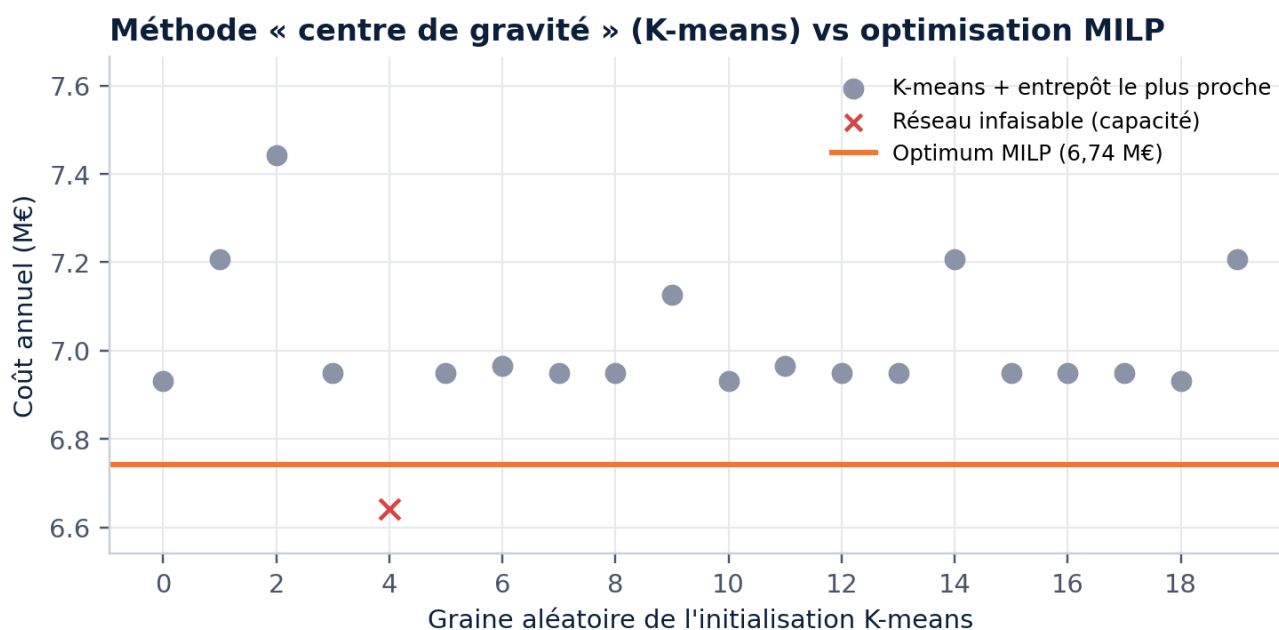


Fig. 6 — Coût du réseau obtenu par K-means selon l'initialisation, comparé à l'optimum MILP (ligne orange).

Sur 20 essais, 1 donnent un réseau **infaisable** (deux centres de gravité tombent sur le même entrepôt, dont la capacité ne suffit pas). Les autres coûtent de 2,8 % à 10,4 % de plus que l'optimum, soit jusqu'à 700 k€ par an. La raison est structurelle : le centre de gravité ne voit ni les coûts fixes, ni les capacités, ni le transport amont depuis Gênes.

3.9 — Scénario : et si les marchandises arrivaient par Hamburg ?

En remplaçant Gênes par le port de Hamburg comme point d'entrée, le réseau optimal devient **Hannover · Erfurt · Nuernberg** et le coût terrestre tombe à 5,24 M€ (-22,2 %). Attention à ne pas conclure trop vite : la traversée maritime Radès → Hamburg est beaucoup plus longue que Radès → Gênes, et son coût n'est pas modélisé ici. Ce scénario montre que **le choix du port est une décision plus structurante que le choix des entrepôts** — et qu'il faudrait l'intégrer au modèle (une variable binaire par port) avant de trancher.

4 Niveau tactique — dimensionnement des lots

4.1 — Le problème

À l'usine de Ben Arous, trois références critiques partagent une **ligne goulot** (centre d'usinage CNC). Chaque lancement de production coûte cher : réglage, premières pièces rebutées, contrôle — c'est le **coût de lancement** S . Produire plusieurs mois d'un coup évite des lancements mais crée du stock, qui coûte h € par palette et par mois (capital immobilisé, surface, risque d'obsolescence). La question : **quand produire, et combien ?**

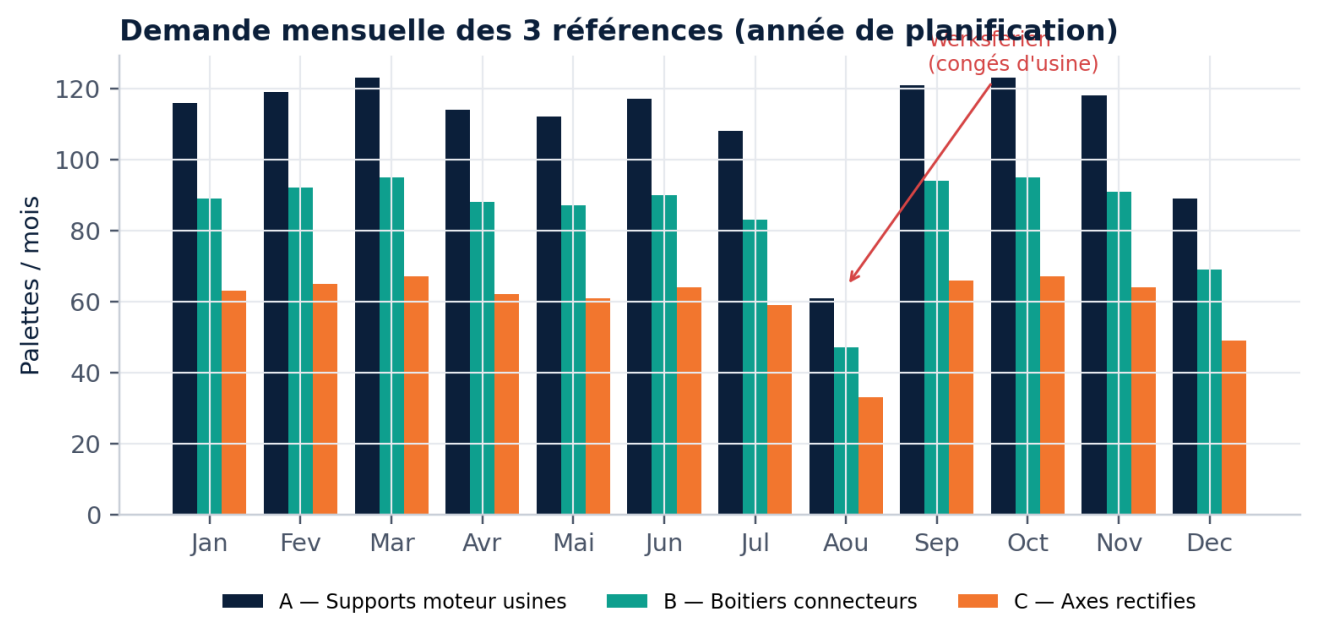


Fig. 7 — Demande mensuelle des trois références. Le creux d'août correspond aux congés des usines allemandes (Werksferien).

Réf.	Désignation	Coût lancement S	Temps réglage	Temps / palette	Stockage h (/pal/mois)	Demande / an
A	Supports moteur usines	4 800 €	14 h	2,20 h	42 €	1 321
B	Boitiers connecteurs	3 600 €	10 h	1,80 h	35 €	1 020
C	Axes rectifiés	4 200 €	12 h	3,00 h	55 €	720

Tableau 3 — Données des trois références. Capacité régulière de la ligne : 630 h/mois, heures supplémentaires jusqu'à 60 h/mois à 45 €/h.

4.2 — Les heuristiques classiques

- **Lot-for-Lot (LFL)** : produire chaque mois exactement la demande du mois. Zéro stock, un lancement par mois. C'est la logique « juste-à-temps » pure.
- **POQ (Periodic Order Quantity)** : calculer la quantité économique de Wilson $Q^* = \sqrt{(2 \cdot D \cdot S / h)}$, la convertir en nombre de mois de couverture, puis produire à intervalle fixe. Suppose une demande régulière.
- **Silver-Meal** : étendre le lot au mois suivant tant que le **coût moyen par mois couvert** diminue. Simple, souvent bon, mais myope : il s'arrête au premier minimum local.

4.3 — Wagner-Whitin : l'optimum par programmation dynamique

Wagner et Whitin (1958) ont démontré une propriété clé : dans une solution optimale, **on ne produit que lorsque le stock est nul**. Chaque lot couvre donc un nombre entier de mois consécutifs, ce qui permet une récurrence :

$$F(0) = 0$$
$$F(k) = \min_{0 \leq t < k} \left[F(t) + S + \sum_{u=t}^{k-1} h(u-t) d_u \right]$$

aucun mois à couvrir

lot lancé au mois t, couvrant t...k-1

$F(k)$ est le coût minimal pour couvrir les k premiers mois. On calcule $F(1)$, $F(2)$... $F(12)$, en mémorisant pour chaque k le mois du dernier lancement ; on remonte ensuite la chaîne pour reconstruire le plan. Complexité : $O(T^2)$, instantané.

FAITES-LE À LA MAIN UNE FOIS — 5 MINUTES

Produit A, janvier à mars : $d = 116, 119, 123$; $S = 4\,800$ € ; $h = 42$ €.

$F(1) = 4\,800$ (un lancement en janvier).

$F(2) = \min(F(1) + 4\,800 = 9\,600 ; 4\,800 + 42 \times 1 \times 119 = 9\,798) = \mathbf{9\,600}$ → on relance en février.

$F(3) = \min(9\,600 + 4\,800 = 14\,400 ; 4\,800 + 4\,800 + 42 \times 123 = 14\,766 ; 4\,800 + 42 \times 119 + 2 \times 42 \times 123 = 20\,130) = \mathbf{14\,400}$.

Lecture : stocker un mois de demande coûte $42 \times 119 \approx 5\,000$ €, à peine plus qu'un lancement (4 800 €). On est au point de bascule — voilà pourquoi, plus loin dans l'année, dès que $h \cdot d$ passe sous S (avril-mai : $42 \times 112 = 4\,704$ €), Wagner-Whitin regroupe deux mois.

4.4 — Le même problème en MILP (et pourquoi c'est utile)

$$\begin{aligned} \min \quad & \sum_{t=1}^T (S y_t + h I_t) && \text{lancements + stockage} \\ I_{t-1} + x_t - d_t = I_t \quad & \forall t, \quad I_0 = 0 && \text{bilan de stock} \\ x_t \leq M_t y_t, \quad & M_t = \sum_{u \geq t} d_u && \text{big-M le plus serré possible} \\ x_t, I_t \geq 0, \quad & y_t \in \{0, 1\} \end{aligned}$$

Pourquoi résoudre en MILP un problème qu'un algorithme de 15 lignes résout déjà ? Parce que (1) les deux méthodes doivent donner le même coût — c'est un **test unitaire** de votre code, comme FT06 dans votre projet 4 ; et (2) le MILP s'étend sans effort aux contraintes que la programmation dynamique ne sait pas gérer : capacité partagée, temps de réglage, heures supplémentaires.

	Lot-for-Lot	POQ	Silver-Meal	Wagner-Whitin	MILP (CBC)
Produit A	57 600 (+6,3 %)	57 600 (+6,3 %)	56 262 (+3,8 %)	54 204	54 204
Produit B	43 200 (+12,4 %)	38 435	38 435	38 435	38 435
Produit C	50 400 (+14,8 %)	43 900	43 900	43 900	43 900

Tableau 4 — Coût annuel (€, *lancements + stockage*) selon la méthode, chaque produit étant planifié seul et sans limite de capacité.

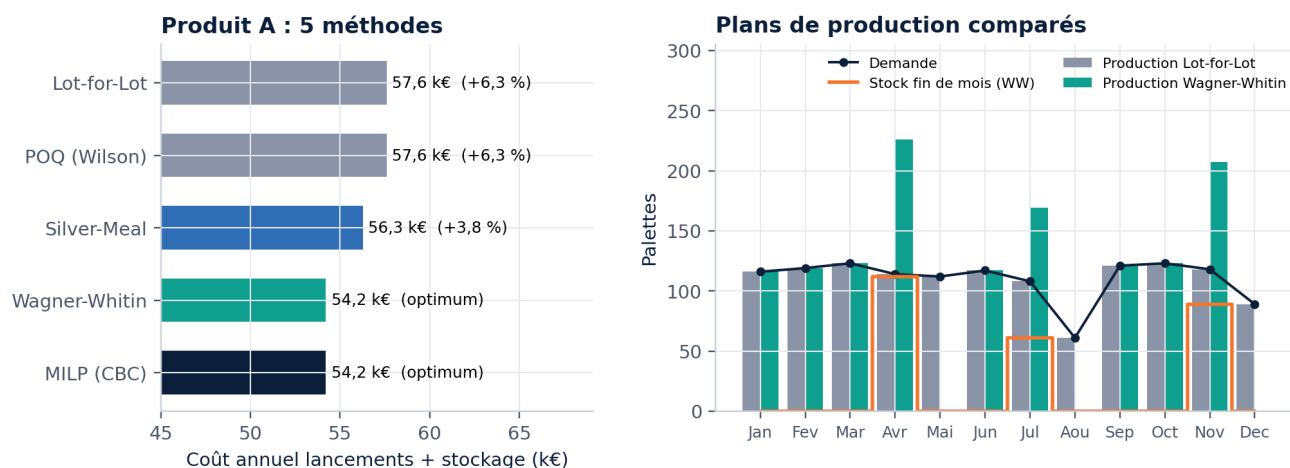


Fig. 8 — Produit A : comparaison des cinq méthodes (gauche) et plans de production Lot-for-Lot vs Wagner-Whitin (droite).

Wagner-Whitin et le MILP trouvent exactement le même coût pour les trois produits : le code est validé. Pour le produit A, l'optimum (54 204 €) utilise 9 lancements au lieu de 12, et Silver-Meal reste 3,8 % au-dessus. POQ se confond ici avec Lot-for-Lot, car la quantité de Wilson (159 palettes) est proche d'un mois de demande — une illustration des limites de l'EOQ quand la demande varie.

4.5 — Le vrai problème : trois produits, une seule ligne (CLSP)

Planifier chaque produit séparément ignore que les trois partagent la même ligne. Le résultat est spectaculaire :

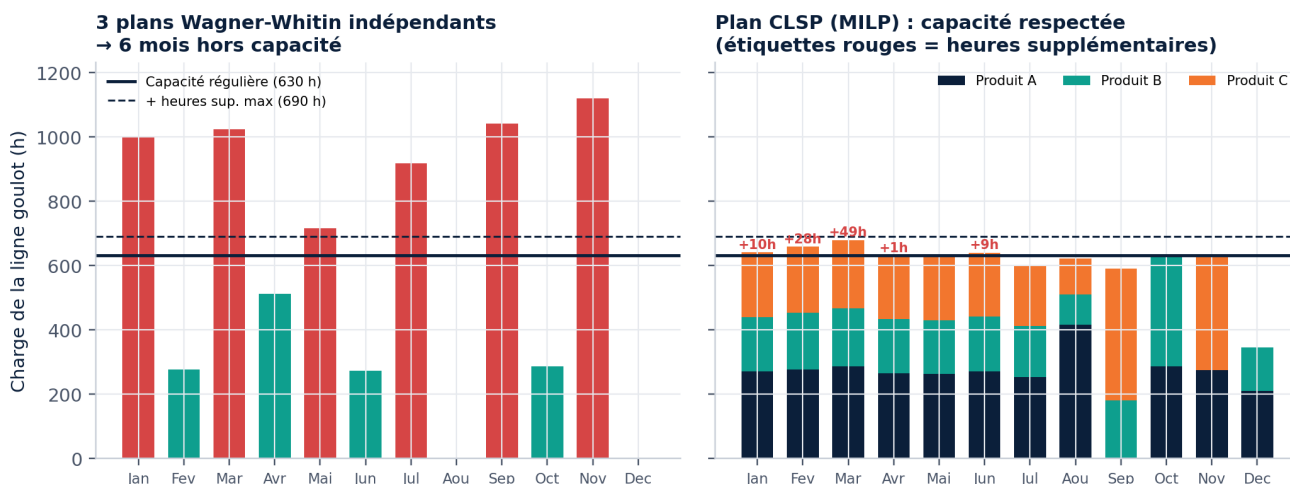


Fig. 9 — Gauche : charge de la ligne si chaque produit suit son plan Wagner-Whitin optimal. Droite : plan du modèle CLSP qui intègre la capacité.

Les plans individuels « optimaux » concentrent les lancements sur les mêmes mois : la charge atteint 1 118 h pour 630 h disponibles, et 6 mois dépassent même la capacité maximale avec heures supplémentaires. **La somme d'optimums locaux n'est pas un optimum global — ici, elle n'est même pas réalisable.** Le modèle CLSP (*Capacitated Lot-Sizing Problem*) corrige cela :

$$\min \sum_k \sum_t (S_k y_{kt} + h_k I_{kt}) + \sum_t c^O O_t$$
$$I_{k,t-1} + x_{kt} - d_{kt} = I_{kt} \quad \forall k, t$$
$$x_{kt} \leq M_{kt} y_{kt} \quad \forall k, t$$
$$\sum_k (a_k x_{kt} + s_k y_{kt}) \leq C + O_t \quad \forall t$$
$$0 \leq O_t \leq O^{\max}, \quad x_{kt} \in \mathbb{N}, \quad y_{kt} \in \{0, 1\}$$

lancements + stock + heures sup.

bilan de stock par produit

lancement obligatoire

capacité : production + réglages

a_k est le temps de production d'une palette, s_k le temps de réglage et O_t les heures supplémentaires du mois t . La nouveauté par rapport à la section 4.4 est la contrainte de capacité, qui couple les trois produits : le temps de réglage d'un produit consomme la capacité des deux autres.

Poste de coût	3 plans Wagner-Whitin indépendants	Plan CLSP (MILP)
Lancements	—	134 400 €
Stockage	—	14 799 €
Heures supplémentaires	—	4 410 € (98 h)
Total	136 539 € — mais irréalisable	153 609 € — faisable
Nombre de lancements	21	32

Tableau 5 — Le « prix de la capacité » : +12,5 % par rapport à la borne irréalisable. Résolution CBC : 0,41 s, 120 variables.

Le plan CLSP utilise le creux d'août pour constituer du stock avant la rentrée de septembre, lisse les lancements pour ne jamais dépasser la ligne, et ne recourt aux heures supplémentaires qu'en début d'année, où il n'existe aucun stock initial pour absorber la demande.

LIRE UNE INFAISABILITÉ

Sans heures supplémentaires, CBC déclare le problème **infaisable**. Ce n'est pas un bug, c'est une information, et elle se vérifie à la main : en janvier, sans stock initial, les trois produits doivent être lancés. Production : $116 \times 2,2 + 89 \times 1,8 + 63 \times 3,0 \approx 604$ h ; réglages : $14 + 10 + 12 = 36$ h ; total 640 h > 630 h.

Face à un modèle infaisable, cherchez toujours la contrainte en cause et **faites le calcul vous-même** : c'est ce qui transforme un message d'erreur en recommandation (« il faut 10 h de plus en janvier, ou un stock de sécurité en décembre »).

4.6 — Combien vaut une heure de capacité ?

Combien vaut une heure de capacité en plus ?

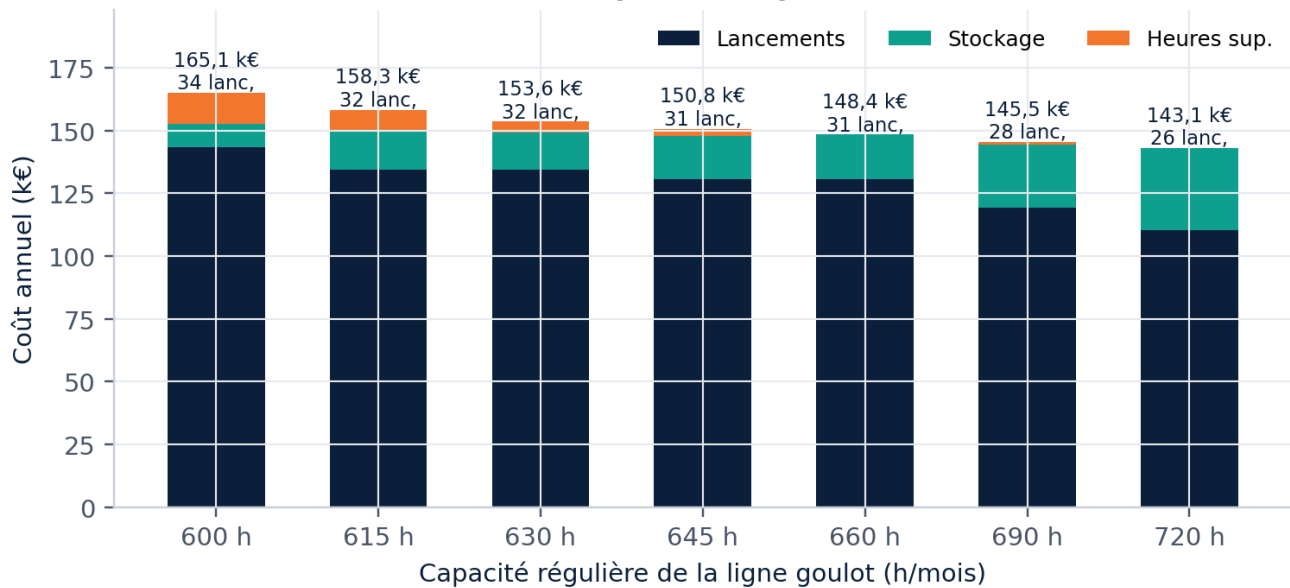


Fig. 10 — Coût annuel optimal du plan en fonction de la capacité régulière de la ligne goulot.

Passer de 630 h à 660 h par mois (+30 h, soit 360 h par an) fait économiser 5 181 € par an, soit environ 14 € par heure de capacité ajoutée. Au-delà, les gains s'effondrent : la capacité n'est plus le facteur limitant, ce sont les coûts de lancement. C'est le raisonnement à tenir face à un directeur d'usine qui hésite à ouvrir un poste de nuit supplémentaire.

5 Niveau opérationnel — tournées de livraison

5.1 — Le problème

Chaque entrepôt livre ses clients deux fois par jour (une vague par poste de travail, logique JIT). Un semi-remorque transporte au plus 33 palettes et une tournée ne peut pas dépasser 700 km (environ 9 h de conduite, limite quotidienne du règlement européen 561/2006). Chaque camion mobilisé coûte 180 € par vague, plus 1,70 € par kilomètre. Le problème de tournées de véhicules avec capacité (*Capacitated Vehicle Routing Problem*, CVRP) cherche l'ensemble de tournées le moins cher.

5.2 — Formulation MILP (2 indices + MTZ)

$$\begin{aligned}
 \min \quad & c^{km} \sum_{(i,j)} d_{ij} x_{ij} + F \sum_{j \in C} x_{0j} && \text{km + camions} \\
 \sum_j x_{ij} = 1, \quad \sum_j x_{ji} = 1 \quad & \forall i \in C && \text{on entre et on sort une fois} \\
 u_j \geq u_i + q_j - Q(1 - x_{ij}) \quad & \forall i \neq j \in C, \quad q_i \leq u_i \leq Q && \text{charge cumulée (MTZ)} \\
 D_j \geq D_i + d_{ij} - 2L(1 - x_{ij}), \quad & D_i + d_{i0} \leq L + L(1 - x_{i0}) && \text{distance cumulée} \leq L \\
 x_{ij} \in \{0, 1\} & &&
 \end{aligned}$$

$x_{ij} = 1$ si un camion va directement de i à j ($0 =$ entrepôt). Les contraintes **MTZ** (Miller-Tucker-Zemlin, 1960) jouent un double rôle : elles comptent la charge cumulée u_i pour respecter la capacité, et elles **interdisent les sous-tours** (des boucles qui ne passent pas par l'entrepôt), car une charge ne peut pas augmenter indéfiniment le long d'un cycle. Le même mécanisme appliqué à la distance cumulée D_i impose la limite de 700 km. Cette formulation est compacte et facile à coder, mais sa relaxation LP est **faible** — vous allez le constater en 5.5.

5.3 — Les heuristiques

- **Livraisons directes** : un camion par client. C'est la référence naïve — et, implicitement, l'hypothèse du coût « par palette·km » du niveau stratégique.
- **Plus proche voisin** : remplir un camion en allant toujours au client faisable le plus proche.
- **Clarke & Wright (1964)** : partir d'un aller-retour par client, puis fusionner les tournées dans l'ordre des plus grandes économies :

$$s_{ij} = d_{0i} + d_{0j} - d_{ij}$$

économie à servir i et j dans la même tournée

L'intuition : si i et j sont proches l'un de l'autre mais loin de l'entrepôt, les servir ensemble évite un long retour. On ne fusionne que par les extrémités des tournées et seulement si la capacité et la limite de distance restent respectées. Soixante ans après, c'est encore la base de nombreux logiciels de tournées.

```

■ level3_vrp.py – cœur de Clarke & Wright
savings = sorted(((d[0][i] + d[0][j] - d[i][j]), i, j)
                  for i in custs for j in custs if i < j), reverse=True)
for s, i, j in savings:
    ri, rj = where[i], where[j]
    if s <= 0 or ri == rj:
        continue
    A, B = routes[ri], routes[rj]
    if A[-1] == i and B[0] == j: merged = A + B          # on colle les extremités
    elif A[0] == i and B[-1] == j: merged = B + A
    elif A[-1] == i and B[-1] == j: merged = A + B[:-1]
    elif A[0] == i and B[0] == j: merged = A[:-1] + B
    else: merged = None                                   # i ou j au milieu : impossible
    if merged and feasible(merged, inst):                  # capacité + 700 km
        routes[ri] = merged; del routes[rj]
        for c in merged: where[c] = ri

```

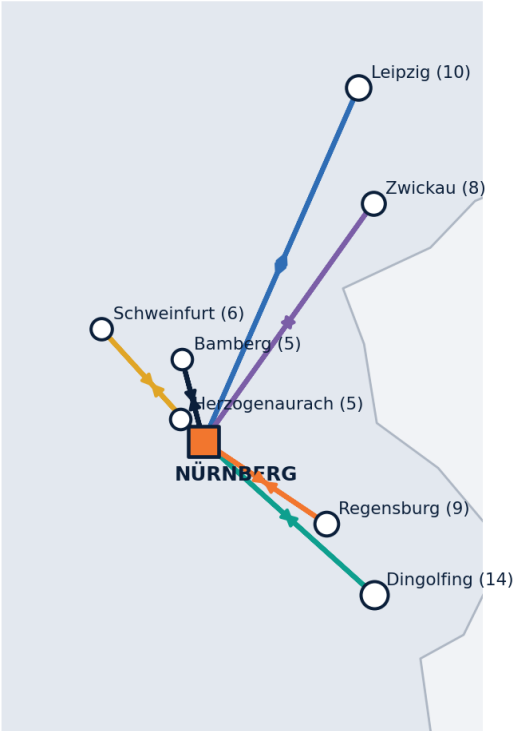
5.4 — Résultats sur les trois entrepôts du réseau initial

Entrepôt	Méthode	Camions	km	Coût / vague	Statut
Kassel	Livraisons directes	7	2 500	5 509 €	heuristique
	Plus proche voisin	4	2 225	4 502 €	heuristique
	Clarke & Wright	3	1 936	3 831 €	heuristique
	MILP (CBC)	3	1 936	3 831 €	optimum prouvé
Nuernberg	Livraisons directes	7	1 974	4 615 €	heuristique
	Plus proche voisin	3	1 490	3 074 €	heuristique
	Clarke & Wright	2	1 086	2 206 €	heuristique
	MILP (CBC)	2	1 086	2 206 €	optimum prouvé
Ulm	Livraisons directes	7	2 442	5 411 €	heuristique

Entrepôt	Méthode	Camions	km	Coût / vague	Statut
	Plus proche voisin	3	1 757	3 527 €	heuristique
	Clarke & Wright	3	1 717	3 459 €	heuristique
	MILP (CBC)	3	1 717	3 459 €	optimum prouvé

Tableau 6 — Une vague de livraison par entrepôt. Lignes surlignées : optimum prouvé.

Livraisons directes
7 camions · 1974 km · 4 615 €/vague



Tournées optimisées (MILP)
2 camions · 1086 km · 2 206 €/vague

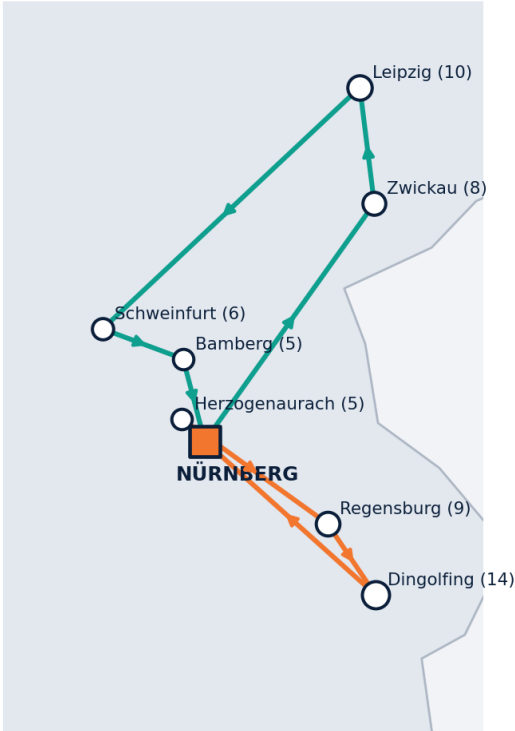


Fig. 11 — Entrepôt de Nuremberg : sept camions en livraisons directes (gauche) contre deux tournées optimales (droite).
Entre parenthèses : palettes par vague.

Les tournées optimales réduisent le coût d'une vague de 30 à 52 %. Sur ces petites instances (7 clients), Clarke & Wright trouve exactement l'optimum que le MILP **prouve** en une seconde environ. Emden, à 346 km de Kassel, reste servi seul : l'aller-retour frôle déjà la limite de 700 km.

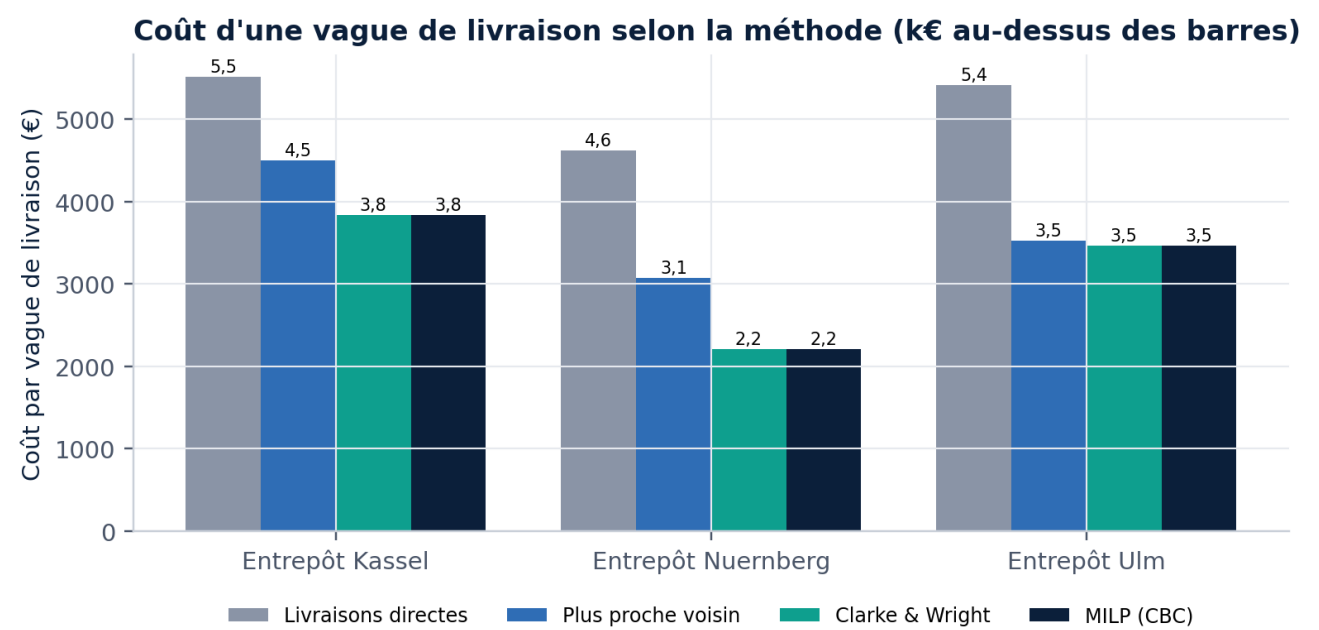


Fig. 12 — Coût d'une vague selon la méthode, pour chaque entrepôt.

5.5 — Passage à l'échelle : quand le MILP décroche

Pour mesurer les limites de la formulation MTZ, on génère des instances aléatoires de 8 à 25 clients autour de Nürnberg (graine fixée) et on donne 30 secondes à CBC, d'abord seul, puis en lui fournissant la solution de Clarke & Wright comme point de départ (*warm start*).

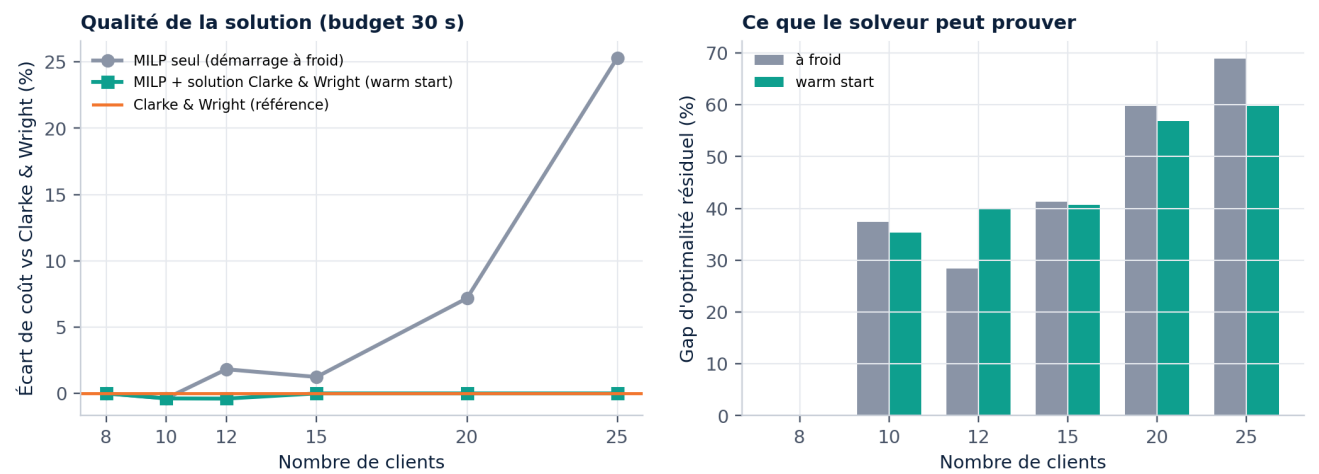


Fig. 13 — Gauche : écart de coût par rapport à Clarke & Wright. Droite : gap d'optimalité que le solveur n'a pas réussi à fermer en 30 s.

Clients	Variables	Clarke & Wright (€)	MILP seul (€)	MILP + warm start (€)	Gap restant
8	88	2 066	2 066 (optimal)	2 066 (optimal)	0 %
10	130	2 628	2 618 (feasible)	2 618 (feasible)	35 %
12	180	2 944	2 997 (feasible)	2 932 (feasible)	40 %
15	270	3 112	3 151 (feasible)	3 112 (feasible)	41 %
20	460	4 658	4 993 (feasible)	4 658 (feasible)	57 %
25	700	5 680	7 118 (feasible)	5 680 (feasible)	60 %

Tableau 7 — Instances aléatoires, budget de 30 s par résolution. « feasible » = solution trouvée mais optimalité non prouvée.

Jusqu'à 8 clients, l'optimum est prouvé en 3 secondes. Dès 10 clients, CBC n'arrive plus à fermer le gap en 30 s ; à 25 clients, le MILP seul rend une solution **25 % plus chère** qu'une heuristique de 1964 qui tourne en une milliseconde. Avec le warm start, le MILP ne fait jamais moins bien que Clarke & Wright et l'améliore parfois légèrement (10 et 12 clients). Mais le gap reste de 35 à 60 % : la borne MTZ est trop faible pour prouver quoi que ce soit.

À RETENIR

Le résultat rejoint celui de votre projet 4 mais va plus loin : **la faiblesse vient de la formulation, pas seulement de la taille**. Les solveurs industriels de tournées utilisent d'autres approches (coupes de capacité par *branch-and-cut*, génération de colonnes, métaheuristiques comme la recherche locale adaptative). En pratique, on combine : une heuristique rapide pour une bonne solution, un solveur exact pour l'améliorer et mesurer la distance à l'optimum.

6 Boucler les niveaux de décision

Phase 4 — validation & intégration

C'est la section la plus importante du guide. Jusqu'ici, chaque niveau a été optimisé dans son coin. Or le niveau stratégique reposait sur une hypothèse — 0,14 € par palette et par kilomètre en aval — que le niveau opérationnel permet maintenant de **vérifier**.

6.1 — Le constat : l'hypothèse aval était largement sous-estimée

En résolvant les tournées de chaque entrepôt, puis en multipliant par le nombre de vagues annuelles, on obtient le coût aval réel. On le divise par le nombre de palettes·km « à vol de camion direct » pour le comparer au tarif supposé :

Organisation des livraisons	Coût aval annuel (tournées)	Coût réel par palette·km	vs hypothèse 0,14 €
1 vague par jour	3,05 M€	0,222 €	×1,6
2 vagues par jour (JIT, retenu)	4,37 M€	0,317 €	×2,3

Tableau 8 — Coût aval issu des tournées optimales, réseau Kassel · Nürnberg · Ulm.

Avec deux vagues par jour, les camions partent à moitié vides pour respecter le juste-à-temps : le coût réel est 2,3 fois l'hypothèse. Le vrai coût du réseau initial n'est donc pas 6,74 M€ mais **9,18 M€** (+36 %). Au passage, on chiffre le **coût du JIT** : passer d'une à deux vagues par jour coûte 1,32 M€ par an — un chiffre à mettre en regard des stocks évités chez le client.

6.2 — La boucle d'itération

1. Résoudre le CFLP avec le tarif aval courant.
2. Pour chaque entrepôt ouvert, résoudre le VRP de ses clients (MILP, 2 vagues/jour).
3. Calculer le tarif aval effectif = coût annuel des tournées / palettes·km.
4. Si le tarif a changé de plus de 2 %, recommencer en 1 avec le nouveau tarif ; sinon, arrêter.

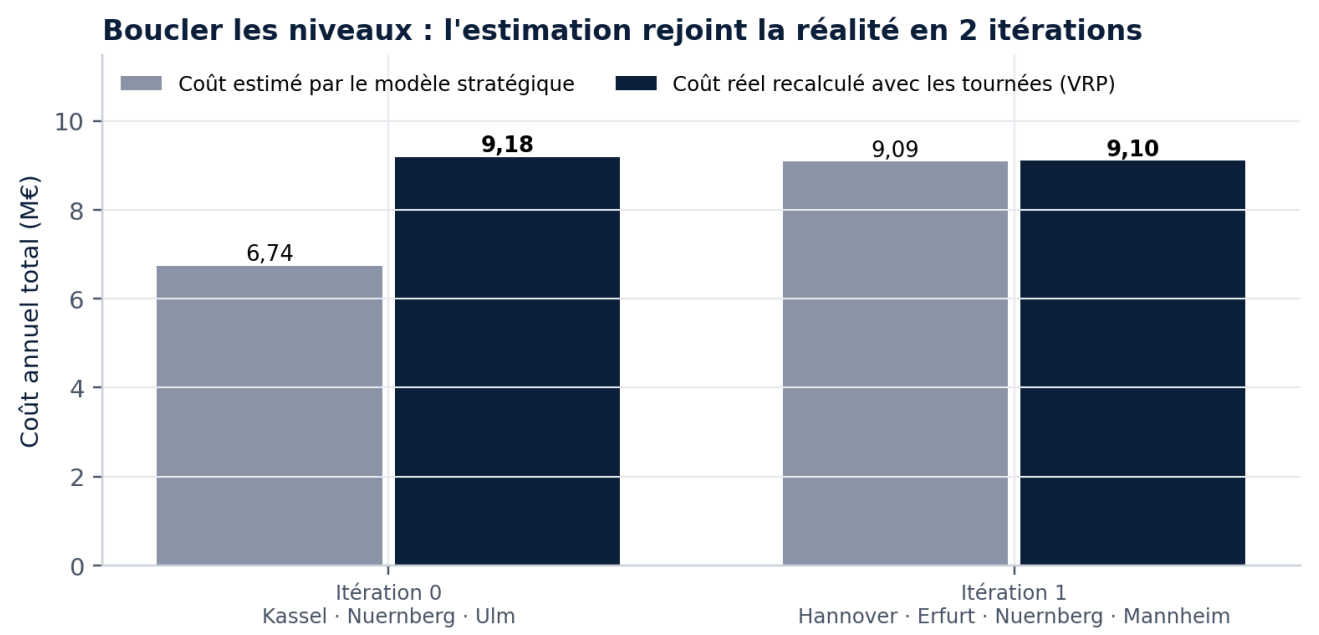


Fig. 14 — Coût estimé par le modèle stratégique et coût réel recalculé avec les tournées, à chaque itération.

Itér.	Tarif aval utilisé	Réseau optimal	Coût estimé	Coût réel	Tarif réel
0	0,140 €	Kassel · Nuernberg · Ulm	6,74 M€	9,18 M€	0,317 €
1	0,317 €	Hannover · Erfurt · Nuernberg · Mannheim	9,09 M€	9,10 M€	0,319 €

La boucle converge en 2 itérations. Avec le tarif recalé, le modèle ouvre **quatre** entrepôts — Hannover, Erfurt, Nuernberg et Mannheim — plus proches des clients, et son estimation (9,09 M€) colle désormais au coût réel (9,10 M€). C'est exactement ce que la carte de robustesse de la section 3.7 annonçait : un transport aval multiplié par deux fait basculer le réseau.

Réseau recommandé — après recalage sur les tournées

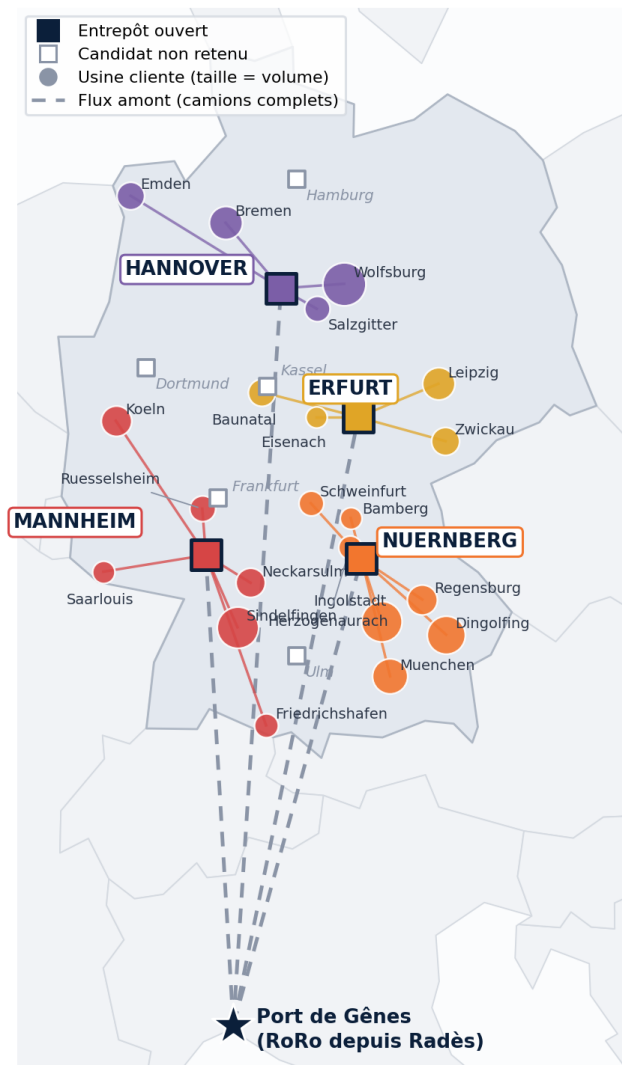


Fig. 15 — Réseau recommandé après recalage : quatre entrepôts plus proches des usines clientes.

6.3 — Ce que cette boucle enseigne

- **L'erreur la plus coûteuse portait sur le niveau du coût, pas sur le choix du réseau.** Les deux réseaux ont un coût réel très proche (9,18 M€ contre 9,10 M€, soit 0,9 % d'écart). Mais un budget construit sur 6,74 M€ aurait été dépassé de 36 %.
- **Quand deux décisions sont à moins de 1 % l'une de l'autre, le modèle ne tranche plus seul.** Il faut ajouter des critères qu'il ignore : disponibilité de main-d'œuvre logistique, risque de saturation, proximité des clients stratégiques, possibilité d'extension. L'optimisation éclaire la décision, elle ne la remplace pas.
- **Un modèle stratégique n'est jamais meilleur que ses hypothèses de coût.** Le réflexe professionnel est de les valider par le niveau inférieur — ou par des données réelles de transport — avant de présenter un résultat à la direction.

EN ENTRETIEN

« En reliant mon modèle de localisation à mon modèle de tournées, j'ai découvert que mon hypothèse de coût de transport était sous-estimée d'un facteur 2,3. J'ai bouclé les deux modèles ; le réseau a convergé en deux itérations, et le coût estimé est devenu cohérent avec le coût réel. » — Cette phrase, chiffres à l'appui, vaut plus que n'importe quel score de modèle.

7 Synthèse type A3

Phase 5 — une page pour tout raconter

1. CONTEXTE

Fournisseur tunisien, 21 usines clientes en Allemagne, 81 900 palettes/an. Entrée par Gênes. Trois décisions à trois horizons : réseau, production, tournées.

2. PROBLÈME

Où ouvrir des entrepôts, quand produire et combien, comment livrer — au moindre coût total et en respectant capacités, JIT et temps de conduite ?

3. ANALYSE

Arbitrages identifiés : coûts fixes vs transport, lancements vs stock, camions vs kilomètres. Heuristiques intuitives testées comme référence.

4. MÉTHODE

3 MILP (CFLP, CLSP, CVRP-MTZ) en PuLP/CBC, validés par programmation dynamique et comparés à K-means, Silver-Meal, Clarke & Wright. Boucle de recalage entre niveaux.

5. RÉSULTATS

Réseau recommandé : Hannover · Erfurt · Nuernberg · Mannheim, 9,10 M€/an. Plan de production faisable à 153,6 k€. Tournées -30 à -52 %. Hypothèse aval corrigée (×2,3).

6. SUITE

Recaler les coûts avec des devis transporteurs réels, intégrer le choix du port et le fret maritime, ajouter les stocks de sécurité et les fenêtres horaires de livraison.

9,10 M€

coût annuel total du réseau
recommandé

+13 %

prix de la capacité (vs plans
irréalisables)

2 vs 7

camions par vague (Nürnberg)

2

itérations pour réconcilier les
niveaux

8 Reproduire et s'approprier le projet

Guide pratique

8.1 — Installation et exécution

■ terminal

```

pip install pulp numpy matplotlib          # le coeur (CBC est inclus dans PuLP)
pip install "geopandas==0.14.4"          # uniquement pour les cartes (fond de carte intègre)

python level1_cflp.py                      # teste le niveau 1 seul (< 1 s)
python level2_lotsizing.py                # teste le niveau 2 seul (< 2 s)
python level3_vrp.py                      # teste le niveau 3 seul (~ 3 s)
python run_all.py --fast                  # toutes les expériences sauf l'échelle VRP (~ 20 s) -> results.json
python scale_test.py 8 10 12 15 20 25    # test d'échelle VRP (~ 6 min)
python charts.py                          # regénère les 15 figures dans figures/

```

8.2 — Structure du dépôt GitHub

■ arborescence

```
supply-chain-milp/
|-- README.md           contexte, 3 niveaux, resultats cles, carte du reseau (le plus important)
|-- data.py             geographie reelle + parametres economiques
|-- level1_cflp.py      localisation d'entrepots (CFLP) + heuristique K-means
|-- level2_lotsizing.py LFL, PQ, Silver-Meal, Wagner-Whitin, ULSP, CLSP
|-- level3_vrp.py       direct, plus proche voisin, Clarke & Wright, MILP-MTZ
|-- run_all.py          toutes les experiences -> results.json
|-- scale_test.py       passage a l'echelle du VRP
|-- charts.py          figures
`-- figures/            cartes et graphiques
```

8.3 — Plan de travail sur 4 semaines

Semaine	Objectif	Livrable de fin de semaine
1	Section 2 + niveau 1 : écrire le CFLP sur papier, puis en PuLP	Réseau optimal + courbe en U
2	Niveau 2 : Wagner-Whitin à la main, puis en code, puis CLSP	Tableau 4 reproduit + plan CLSP
3	Niveau 3 : Clarke & Wright, puis MILP-MTZ, puis test d'échelle	Carte des tournées + tableau 7
4	Boucle d'intégration, README, pitch, exercices	Dépôt GitHub publié + pitch chronométré

8.4 — Exercices progressifs

Faites-les dans l'ordre, sans lire l'indice au premier essai. Chacun fait travailler une compétence précise et prolonge naturellement le code fourni.

EXERCICE 1 — FACILE · MODIFIER UNE DONNÉE

Réduisez la capacité d'Ulm à 28 000 palettes. Quel réseau devient optimal ? De combien le coût augmente-t-il ?

Indice — Modifiez la ligne d'Ulm dans DEPOTS (data.py) puis relancez `solve_cflp()`. Compétence : sensibilité.

EXERCICE 2 — FACILE · AJOUTER UNE CONTRAINTE LOGIQUE

Emden et Bremen appartiennent au même groupe client et exigent le même entrepôt. Imposez-le.

Indice — `for j in J: m += x[e][j] == x[b][j]` (e, b = indices des deux clients). Compétence : contraintes de couplage.

EXERCICE 3 — MOYEN · RETARDS AUTORISÉS

Dans le CLSP, autorisez les retards de livraison (backlog) à 150 €/palette/mois, sans heures supplémentaires. Le modèle redevient-il faisable ? Que choisit-il ?

Indice — Ajoutez `B[k][t] >= 0` ; bilan : `I[k][t-1] - B[k][t-1] + x[k][t] - d[k][t] == I[k][t] - B[k][t]` ; imposez `B[k][11] == 0`.

EXERCICE 4 — MOYEN · AMÉLIORER UNE HEURISTIQUE

Codez la recherche locale 2-opt (inverser un segment d'une tournée si cela raccourcit le trajet) et appliquez-la aux tournées de Clarke & Wright sur les instances de 15 à 25 clients.

Indice — Pour chaque paire (a, b) de positions, tester `route[:a] + route[a:b][::-1] + route[b:]`.
Compétence : métaheuristiques.

EXERCICE 5 — AVANCÉ · CHANGER DE SOLVEUR

Installez `highspy` et résolvez le VRP à 15 clients avec `pulp.HiGHS(timeLimit=30)`. Comparez solution, borne et gap avec CBC.

Indice — Même modèle, une seule ligne change. Compétence : benchmarking de solveurs, lecture de journaux.

EXERCICE 6 — AVANCÉ · INTÉGRER LE CHOIX DU PORT

Ajoutez une variable binaire par port (Gênes, Hamburg) avec un coût maritime par palette que vous estimerez, et laissez le CFLP choisir le port et les entrepôts simultanément.

Indice — Indexez le transport amont par port : `c[i, j, p]` ; ajoutez `z[p]` binaire et `x[i][j][p] <= z[p]`.

8.5 — Checklist des compétences acquises

Compétence
☐ Traduire un problème métier en ensembles, variables, objectif et contraintes
☐ Modéliser des décisions oui/non avec des binaires et un big-M serré
☐ Expliquer relaxation LP, branch-and-bound, borne et gap d'optimalité
☐ Distinguer une formulation forte d'une formulation faible, et le mesurer
☐ Lire correctement un statut de solveur (piège « Optimal » de PuLP)
☐ Programmer une récurrence de programmation dynamique (Wagner-Whitin)
☐ Coder et comparer des heuristiques constructives (Silver-Meal, Clarke & Wright)
☐ Accélérer un MILP avec une solution de départ (warm start)
☐ Interpréter une infaisabilité et la transformer en recommandation
☐ Mener une analyse de sensibilité et de robustesse d'une décision
☐ Relier des niveaux de décision et recaler une hypothèse par un modèle aval

9 Présenter ce projet en entretien

Phase 6 — valorisation

9.1 — Le pitch de 2 minutes, en trois langues

FRANÇAIS

« J'ai modélisé la supply chain d'un fournisseur automobile tunisien qui livre 21 usines en Allemagne, sur trois niveaux de décision. Au niveau stratégique, un MILP de localisation choisit les entrepôts ; au niveau tactique, un modèle de lot-sizing avec capacité planifie la production ; au niveau opérationnel, un modèle de tournées organise les livraisons. J'ai comparé chaque modèle à des heuristiques classiques : par exemple, planifier les produits séparément surchargeait la ligne de 78 %. Le résultat le plus important est venu en reliant les niveaux : le coût réel des tournées était 2,3 fois plus élevé que mon hypothèse ; après deux itérations, le réseau recommandé est passé à quatre entrepôts et l'estimation est devenue cohérente. J'en retiens qu'un modèle n'est jamais meilleur que ses hypothèses, et qu'il faut les valider par le niveau inférieur. »

ENGLISH

"I modelled the supply chain of a Tunisian automotive supplier serving 21 plants in Germany, across three decision levels: a facility-location MILP for the warehouse network, a capacitated lot-sizing model for production, and a vehicle-routing model for daily deliveries. I benchmarked every model against classic heuristics — planning each product separately, for instance, overloaded the bottleneck line by 78%. The key insight came from linking the levels: actual routing costs were 2.3 times my initial assumption. After two iterations the recommended network moved to four warehouses and the cost estimate became consistent. My takeaway: a model is only as good as its assumptions, and the lower planning level is how you validate them."

DEUTSCH (NIVEAU B1)

„Ich habe die Lieferkette eines tunesischen Automobilzulieferers modelliert, der 21 Werke in Deutschland beliefert — auf drei Planungsebenen: Standortplanung der Lager, Losgrößenplanung in der Produktion und Tourenplanung für die Auslieferung, jeweils als gemischt-ganzzahliges Optimierungsmodell in Python. Ich habe jedes Modell mit klassischen Heuristiken verglichen. Das wichtigste Ergebnis: Die echten Transportkosten waren 2,3-mal höher als meine Annahme. Nach zwei Iterationen war das Modell konsistent. Ich habe gelernt: Ein Modell ist nur so gut wie seine Annahmen.“

9.2 — Questions probables et pistes de réponse

Question probable	Point clé de la réponse
Pourquoi PuLP et CBC plutôt que Gurobi ?	Gratuits, reproductibles par n'importe qui, suffisants pour ces tailles. Le modèle est indépendant du solveur : passer à Gurobi, c'est une ligne. Sur les VRP plus grands, un solveur commercial fermerait mieux le gap — mais le problème principal est la formulation MTZ, pas le solveur.
Pourquoi MILP ici, et CP-SAT dans votre projet d'ordonnancement ?	Le CFLP et le lot-sizing sont des problèmes de flux avec des coûts fixes : leur relaxation LP est informative, le MILP y excelle. L'ordonnancement repose sur des contraintes disjonctives, que CP-SAT gère nativement avec NoOverlap.
Comment savez-vous que vos solutions sont bonnes ?	Trois niveaux de preuve : optimum prouvé par le solveur (statut vérifié via <code>sol_status</code>), validation croisée (Wagner-Whitin = MILP), et gap d'optimalité chiffré quand l'optimum n'est pas prouvé.
Vos données ne sont pas réelles...	La géographie l'est ; les coûts sont des ordres de grandeur construits. La démarche — et en particulier la boucle de recalage — est précisément ce qu'on applique quand les données arrivent. Sur mon PFA chez Tunistools, j'ai déjà travaillé avec 114 000 enregistrements de production réels.
Et avec 2 000 clients ?	Découper par zones (clustering), résoudre les tournées avec des heuristiques ou OR-Tools Routing, garder le MILP pour les décisions agrégées, et utiliser le warm start pour combiner les deux.
Qu'est-ce que vous changeriez en premier ?	Intégrer le choix du port avec le fret maritime, puis rendre la demande incertaine (scénarios) : un réseau se décide pour dix ans, il doit être robuste, pas seulement optimal pour une année.

9.3 — Vocabulaire allemand utile

Français	Allemand
Localisation d'entrepôts	Standortplanung
Dimensionnement des lots	Losgrößenplanung
Tournées de véhicules	Tourenplanung
Chaîne logistique	Lieferkette / Supply Chain
Programmation linéaire en nombres entiers	Gemischt-ganzzahlige lineare Optimierung (MILP)
Fonction objectif / contrainte	Zielfunktion / Nebenbedingung
Variable de décision	Entscheidungsvariable
Coût de lancement / temps de réglage	Rüstkosten / Rüstzeit
Coût de stockage	Lagerhaltungskosten
Heures supplémentaires	Überstunden
Camion complet / groupage	Komplettladung (FTL) / Teilladung (LTL)
Méthode des économies	Savings-Verfahren (Clarke & Wright)
Gap d'optimalité	Optimalitätslücke
Congés d'usine	Werksferien

10 Limites & prolongements

Honnêteté méthodologique

- **Données économiques construites** : volumes et coûts sont des ordres de grandeur ; les conclusions chiffrées sont à recaler avec des devis de transporteurs et des coûts d'entrepôts réels.
- **Demande déterministe** : aucune incertitude n'est modélisée ; un réseau décidé pour dix ans devrait être testé sur plusieurs scénarios de demande (optimisation stochastique ou robuste).
- **Fret maritime ignoré** : le port de Gênes est imposé ; le scénario Hamburg montre que ce choix est structurant.
- **Pas de stock de sécurité** ni de délais de transport dans le lot-sizing : l'usine et les entrepôts sont planifiés comme si tout arrivait à l'heure.
- **Tournées simplifiées** : pas de fenêtres horaires de livraison (créneaux imposés par les usines), temps de conduite approché par une limite kilométrique, flotte homogène.
- **Formulation MTZ faible** : suffisante pour 7 clients par entrepôt, insuffisante au-delà de 10.

Prolongements naturels

- **Réseau multi-périodes** : ouvrir des entrepôts progressivement sur 5 ans avec une demande croissante.
- **Stocks multi-échelons** : dimensionner les stocks de sécurité à l'usine et dans les entrepôts (modèle de service garanti).
- **Tournées avec fenêtres horaires** (VRPTW) avec OR-Tools Routing, très utilisé en industrie.
- **Objectif CO₂** : ajouter le coût carbone du transport (péage CO₂ allemand) et tracer le front coût / émissions.
- **Lien avec vos autres projets** : utiliser la prévision de votre PFA comme demande du lot-sizing, ou la maintenance prédictive pour réduire la capacité disponible de la ligne goulot.

DERNIER CONSEIL

Ne cherchez pas à tout faire. Un projet terminé, documenté, avec des résultats chiffrés honnêtes et un pitch maîtrisé vaut plus qu'un projet deux fois plus ambitieux jamais fini. Les trois niveaux de ce guide, plus la boucle d'intégration, constituent déjà un livrable solide.

A Annexe — code source complet

Reproductible à l'identique

Les fichiers ci-dessous reproduisent tous les chiffres de ce guide. Ils sont également fournis dans l'archive ZIP qui accompagne ce document, avec le script de figures `charts.py`.

A.1 — data.py

```
■ data.py
"""
data.py – Données du cas d'étude (Projet 5 : supply chain par MILP)

Géographie REELLE (coordonnées GPS des sites automobiles allemands),
paramètres économiques CONSTRUITS sur des ordres de grandeur crédibles
(entreprise fictive "TunAuto Components", Ben Arous, Tunisie).
"""
import math

# -----
# 1. Point d'entrée en Europe : port de Genes (RoRo depuis Rades)
# -----
PORT = ("Genes (port)", 44.41, 8.93)

# -----
# 2. Usines clientes (OEM + équipementiers Tier-1) : nom, lat, lon, demande
#   Demande = palettes / an
# -----
CLIENTS = [
    # --- OEM ---
    ("Wolfsburg (VW)", 52.42, 10.79, 7800),
    ("Emden (VW)", 53.37, 7.21, 3200),
    ("Bremen (Mercedes)", 53.08, 8.80, 4600),
    ("Koeln (Ford)", 50.94, 6.96, 3900),
    ("Saarlouis (Ford)", 49.31, 6.75, 2100),
    ("Ruesselsheim (Opel)", 49.99, 8.41, 2800),
    ("Sindelfingen (Mercedes)", 48.71, 9.00, 7200),
    ("Neckarsulm (Audi)", 49.19, 9.22, 3600),
    ("Ingolstadt (Audi)", 48.77, 11.42, 6900),
    ("Muenchen (BMW)", 48.18, 11.56, 5200),
    ("Dingolfing (BMW)", 48.63, 12.50, 6100),
    ("Regensburg (BMW)", 49.01, 12.10, 3800),
    ("Leipzig (BMW/Porsche)", 51.34, 12.37, 4400),
    ("Zwickau (VW)", 50.72, 12.49, 3300),
    ("Eisenach (Opel)", 50.98, 10.32, 1900),
    # --- Tier-1 ---
    ("Schweinfurt (ZF)", 50.05, 10.23, 2600),
    ("Herzogenaurach (Schaeffler)", 49.57, 10.89, 2300),
```

```

    ("Bamberg (Bosch)",      49.89, 10.90, 2000),
    ("Friedrichshafen (ZF)", 47.65,  9.48, 2400),
    ("Salzgitter (VW)",      52.15, 10.33, 2700),
    ("Baunatal (VW)",        51.25,  9.41, 3100),
]

# -----
# 3. Entrepôts candidats (hubs logistiques) :
#   nom, lat, lon, cout fixe annuel [EUR], capacite [pal/an], manutention [EUR/pal]
# -----
DEPOTS = [
    ("Hamburg",    53.55, 10.00, 520_000, 45_000, 6.5),
    ("Hannover",   52.37,  9.73, 430_000, 40_000, 5.8),
    ("Dortmund",   51.51,  7.47, 440_000, 40_000, 5.8),
    ("Kassel",     51.31,  9.50, 380_000, 35_000, 5.2),
    ("Frankfurt",  50.11,  8.68, 560_000, 45_000, 6.8),
    ("Erfurt",     50.98, 11.03, 350_000, 35_000, 4.6),
    ("Nuernberg",  49.45, 11.08, 450_000, 40_000, 5.6),
    ("Mannheim",   49.49,  8.47, 470_000, 40_000, 5.9),
    ("Ulm",        48.40,  9.99, 400_000, 35_000, 5.4),
]

# -----
# 4. Parametres de transport
# -----
CIRCUITY = 1.25          # facteur route / vol d'oiseau (valeur standard Europe)
TRUCK_COST_KM = 1.70     # EUR / km pour un semi-remorque complet (FTL)
PALLETS_PER_TRUCK = 33   # palettes europeennes par semi-remorque
INBOUND_EUR_PAL_KM = TRUCK_COST_KM / PALLETS_PER_TRUCK      # ~0.052 (camions pleins)
OUTBOUND_EUR_PAL_KM = 0.14 # livraisons JIT, camions moins remplis -> plus cher

def haversine_km(lat1, lon1, lat2, lon2):
    """Distance orthodromique (vol d'oiseau) en km."""
    r = 6371.0
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp, dl = p2 - p1, math.radians(lon2 - lon1)
    a = math.sin(dp / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl / 2) ** 2

```

```

    return 2 * r * math.asin(math.sqrt(a))

def road_km(a, b):
    """Distance routiere estimee = vol d'oiseau x facteur de detour."""
    return CIRCUIITY * haversine_km(a[1], a[2], b[1], b[2])

# -----
# 5. Donnees du niveau tactique (lot-sizing, usine de Ben Arous)
# -----
MONTHS = ["Jan", "Fev", "Mar", "Avr", "Mai", "Jun",
          "Jul", "Aou", "Sep", "Oct", "Nov", "Dec"]
# Saisonnalite : conges d'usine allemands (Werksferien) en aout, fetes en decembre
SEASONALITY = [1.05, 1.08, 1.12, 1.04, 1.02, 1.06, 0.98, 0.55, 1.10, 1.12, 1.07, 0.81]

# 3 references critiques qui partagent la ligne goulot (centre d'usinage CNC)
# Demande en palettes/mois (avant saisonnalite)
PRODUCTS = {
    # nom: demande de base, cout de lancement [EUR], temps de reglage [h],
    #      temps unitaire [h/palette], cout de stockage [EUR/palette/mois]
    "A": dict(label="Supports moteur usines", base=110, setup_cost=4800,
              setup_time=14, unit_time=2.2, hold_cost=42.0),
    "B": dict(label="Boitiers connecteurs", base=85, setup_cost=3600,
              setup_time=10, unit_time=1.8, hold_cost=35.0),
    "C": dict(label="Axes rectifies", base=60, setup_cost=4200,
              setup_time=12, unit_time=3.0, hold_cost=55.0),
}
LINE_HOURS_PER_MONTH = 630 # capacite reguliere de la ligne goulot [h/mois]
OVERTIME_MAX = 60 # heures supplementaires max par mois
OVERTIME_COST = 45.0 # EUR / heure supplementaire (surcout main-d oeuvre + energie)

def monthly_demand(product):
    base = PRODUCTS[product]["base"]
    return [round(base * s) for s in SEASONALITY]

# -----

# 6. Donnees du niveau operationnel (tournees)
# -----
WORKING_DAYS = 230
VRP_TRUCK_CAP = 33 # palettes
VRP_MAX_KM = 900 # borne de securite (conduite : reglement CE 561/2006, 9h/jour)
VRP_FIXED_PER_TRUCK = 180 # EUR/jour par camion mobilise (chauffeur + amortissement)

```

A.2 — level1_cflp.py

■ level1_cflp.py

```

"""
level1_cflp.py – NIVEAU STRATEGIQUE
Localisation d'entrepôts avec capacités (CFLP, Capacitated Facility Location Problem)

Question : quels entrepôts ouvrir, et quel entrepôt livre quel client,
pour minimiser coûts fixes + manutention + transport amont + transport aval ?
"""

import time
import pulp
from data import (PORT, CLIENTS, DEPOTS, road_km,
                  INBOUND_EUR_PAL_KM, OUTBOUND_EUR_PAL_KM)

I = range(len(CLIENTS)) # clients
J = range(len(DEPOTS))  # entrepôts candidats

def unit_cost(i, j, out_mult=1.0, port=PORT):
    """Coût variable [EUR/palette] pour servir le client i via l'entrepôt j."""
    inbound = road_km(port, DEPOTS[j]) * INBOUND_EUR_PAL_KM
    handling = DEPOTS[j][5]
    outbound = road_km(DEPOTS[j], CLIENTS[i]) * OUTBOUND_EUR_PAL_KM * out_mult
    return inbound + handling + outbound

def solve_cflp(single_sourcing=True, p=None, relax=False, strong=True,
               fixed_mult=1.0, cap_mult=1.0, out_mult=1.0, port=PORT, msg=False):
    """
    single_sourcing : chaque client est livré par UN seul entrepôt (x binaire)
    p                : si fourni, impose exactement p entrepôts ouverts
    relax            : relaxation continue (toutes les variables dans [0,1])
    strong           : ajoute les contraintes de liaison  $x_{ij} \leq y_j$  (formulation forte)
    """
    cat_y = pulp.LpContinuous if relax else pulp.LpBinary
    cat_x = pulp.LpBinary if (single_sourcing and not relax) else pulp.LpContinuous

    m = pulp.LpProblem("CFLP", pulp.LpMinimize)
    y = pulp.LpVariable.dicts("open", J, 0, 1, cat_y)
    x = pulp.LpVariable.dicts("assign", (I, J), 0, 1, cat_x)

```

```

d = [c[3] for c in CLIENTS]
f = [dep[3] * fixed_mult for dep in DEPOTS]
cap = [dep[4] * cap_mult for dep in DEPOTS]
c = {(i, j): unit_cost(i, j, out_mult, port) for i in I for j in J}

# Objectif
fixed = pulp.lpSum(f[j] * y[j] for j in J)
variable = pulp.lpSum(d[i] * c[i, j] * x[i][j] for i in I for j in J)
m += fixed + variable

# (1) chaque client est servi a 100 %
for i in I:
    m += pulp.lpSum(x[i][j] for j in J) == 1, f"demand_{i}"
# (2) capacite : seul un entrepot ouvert peut expedier
for j in J:
    m += pulp.lpSum(d[i] * x[i][j] for i in I) <= cap[j] * y[j], f"cap_{j}"
# (3) liaison forte (redondante en entier, mais resserre la relaxation LP)
if strong:
    for i in I:
        for j in J:
            m += x[i][j] <= y[j], f"link_{i}_{j}"
# (4) nombre d'entrepots impose (analyse de sensibilite)
if p is not None:
    m += pulp.lpSum(y[j] for j in J) == p, "n_open"

t0 = time.perf_counter()
status = m.solve(pulp.PULP_CBC_CMD(msg=msg, timeLimit=120))
dt = time.perf_counter() - t0

res = dict(status=pulp.LpStatus[status], time_s=dt,
            total=pulp.value(m.objective),
            fixed=pulp.value(fixed), variable=pulp.value(variable),
            n_vars=len(m.variables()), n_cons=len(m.constraints))
if res["status"] != "Optimal":
    return res

res["open"] = [DEPOTS[j][0] for j in J if y[j].value() > 1e-6]

```

```

res["y"] = {DEPOTS[j][0]: y[j].value() for j in J}
assign = {}
for i in I:
    shares = {DEPOTS[j][0]: x[i][j].value() for j in J if x[i][j].value() > 1e-6}
    assign[CLIENTS[i][0]] = shares
res["assign"] = assign
# decomposition des couts
inb = sum(d[i] * x[i][j].value() * road_km(port, DEPOTS[j]) * INBOUND_EUR_PAL_KM
          for i in I for j in J)
hdl = sum(d[i] * x[i][j].value() * DEPOTS[j][5] for i in I for j in J)
outb = sum(d[i] * x[i][j].value() * road_km(DEPOTS[j], CLIENTS[i]) * OUTBOUND_EUR_PAL_KM
          * out_mult for i in I for j in J)
res.update(inbound=inb, handling=hdl, outbound=outb)
load = {DEPOTS[j][0]: sum(d[i] * x[i][j].value() for i in I) for j in J}
res["load"] = load
res["util"] = {k: load[k] / (DEPOTS[dd[0] for dd in DEPOTS].index(k))[4] * cap_mult)
               for k in res["open"]}
# distance moyenne ponderee client
res["avg_out_km"] = sum(d[i] * x[i][j].value() * road_km(DEPOTS[j], CLIENTS[i])
                        for i in I for j in J) / sum(d)

return res

if __name__ == "__main__":
    r = solve_cflp()
    print(r["status"], round(r["total"]), r["open"], round(r["time_s"], 2))
    for k, v in r["assign"].items():
        print(f" {k:28s} <- {v}")
    print({k: round(v) for k, v in r["load"].items() if v > 0})

# -----
# Heuristique de reference : centre de gravite par K-Means pondere
# (methode "geographique" intuitive : ignore les couts fixes et le transport amont)
# -----
def kmeans_heuristic(k=3, seed=0, iters=100):
    import random
    from data import haversine_km

```

```

rng = random.Random(seed)
pts = [(c[1], c[2], c[3]) for c in CLIENTS]
cents = [pts[i][:2] for i in rng.sample(range(len(pts)), k)]
for _ in range(iters):
    groups = [[] for _ in range(k)]
    for (la, lo, w) in pts:
        g = min(range(k), key=lambda g: haversine_km(la, lo, *cents[g]))
        groups[g].append((la, lo, w))
    new = []
    for g in range(k):
        if not groups[g]:
            new.append(cents[g]); continue
        W = sum(p[2] for p in groups[g])
        new.append((sum(p[0]*p[2] for p in groups[g]) / W,
                     sum(p[1]*p[2] for p in groups[g]) / W))
    if new == cents:
        break
    cents = new
# "snap" : chaque centre de gravite -> entrepot candidat le plus proche
chosen = []
for (la, lo) in cents:
    j = min(J, key=lambda j: haversine_km(la, lo, DEPOTS[j][1], DEPOTS[j][2]))
    if DEPOTS[j][0] not in chosen:
        chosen.append(DEPOTS[j][0])
return cents, chosen

def evaluate_fixed_network(open_names, single_sourcing=True):
    """Affectation optimale des clients quand le reseau est IMPOSE (y fixes)."""
    m = pulp.LpProblem("assign_fixed", pulp.LpMinimize)
    Jo = [j for j in J if DEPOTS[j][0] in open_names]
    cat = pulp.LpBinary if single_sourcing else pulp.LpContinuous
    x = pulp.LpVariable.dicts("x", (I, Jo), 0, 1, cat)
    d = [c[3] for c in CLIENTS]
    m += (pulp.lpSum(DEPOTS[j][3] for j in Jo)
          + pulp.lpSum(d[i] * unit_cost(i, j) * x[i][j] for i in I for j in Jo))
    for i in I:
        m += pulp.lpSum(x[i][j] for j in Jo) == 1

    for j in Jo:
        m += pulp.lpSum(d[i] * x[i][j] for i in I) <= DEPOTS[j][4]
    st = m.solve(pulp.PULP_CBC_CMD(msg=False))
    return pulp.LpStatus[st], pulp.value(m.objective)

```

A.3 — level2_lotsizing.py

```

■ level2_lotsizing.py
"""
level2_lotsizing.py – NIVEAU TACTIQUE
Dimensionnement des lots de production (lot-sizing) sur 12 mois

Partie A : 1 produit, sans capacite -> Wagner-Whitin (programmation dynamique)
           vs heuristiques (Lot-for-Lot, POQ, Silver-Meal) vs MILP (validation)
Partie B : 3 produits, ligne goulot partagee -> CLSP en MILP
           (capacite + temps de reglage + heures supplementaires)
"""

import math
import time
import pulp
from data import PRODUCTS, MONTHS, LINE_HOURS_PER_MONTH, OVERTIME_MAX, \
    OVERTIME_COST, monthly_demand

T = range(12)

# =====
# Outil commun : evaluer le cout d'un plan de production
# =====
def plan_cost(demand, prod, S, h):
    inv, cost, n_setups = 0, 0.0, 0
    stock = []
    for t in range(len(demand)):
        if prod[t] > 0:
            cost += S
            n_setups += 1
        inv += prod[t] - demand[t]
        assert inv >= -1e-9, "rupture de stock : plan infaisable"
        cost += h * inv
        stock.append(inv)
    return cost, n_setups, stock

def lots_to_plan(demand, lots):
    """lots = liste de (periode_de_production, derniere_periode_couverte)."""
    prod = [0] * len(demand)

```

```

    for (t, k) in lots:
        prod[t] = sum(demand[t:k + 1])
    return prod

# =====
# PARTIE A – heuristiques classiques
# =====
def lot_for_lot(demand):
    return list(demand)

def poq(demand, S, h):
    """Periodic Order Quantity : intervalle derive de l'EOQ (Wilson)."""
    D_mean = sum(demand) / len(demand)
    eoq = math.sqrt(2 * D_mean * S / h)
    k = max(1, round(eoq / D_mean)) # nb de periodes couvertes par un lot
    lots = [(t, min(t + k - 1, len(demand) - 1)) for t in range(0, len(demand), k)]
    return lots_to_plan(demand, lots), eoq, k

def silver_meal(demand, S, h):
    """Etend le lot tant que le cout moyen par periode diminue."""
    n, t, lots = len(demand), 0, []
    while t < n:
        best_k, best_avg, hold = t, S, 0.0
        for k in range(t + 1, n):
            hold += h * (k - t) * demand[k]
            avg = (S + hold) / (k - t + 1)
            if avg > best_avg:
                break
            best_k, best_avg = k, avg
        lots.append((t, best_k))
        t = best_k + 1
    return lots_to_plan(demand, lots)

# =====

```

```

# PARTIE A – Wagner-Whitin (programmation dynamique, optimum garanti)
# =====
def wagner_whitin(demand, S, h):
    """
    F[k] = cout minimal pour satisfaire les demandes des periodes 0..k-1
    F[k] = min_{t<k} ( F[t] + S + sum_{u=t}^{k-1} h*(u-t)*d_u )
    Propriete cle (Wagner & Whitin, 1958) : on ne produit que quand le stock
    est nul -> chaque lot couvre un nombre ENTIER de periodes consecutives.
    """
    n = len(demand)
    F = [0.0] + [math.inf] * n
    pred = [0] * (n + 1)
    for k in range(1, n + 1):
        for t in range(k):
            hold = sum(h * (u - t) * demand[u] for u in range(t, k))
            c = F[t] + S + hold
            if c < F[k]:
                F[k], pred[k] = c, t
    lots, k = [], n
    while k > 0:
        t = pred[k]
        lots.append((t, k - 1))
        k = t
    lots.reverse()
    return lots_to_plan(demand, lots), F[n]

# =====
# PARTIE A – le meme probleme en MILP (validation croisee du DP)
# =====
def uncapacitated_milp(demand, S, h):
    m = pulp.LpProblem("ULSP", pulp.LpMinimize)
    x = pulp.LpVariable.dicts("prod", T, lowBound=0)
    I = pulp.LpVariable.dicts("stock", T, lowBound=0)
    y = pulp.LpVariable.dicts("setup", T, cat=pulp.LpBinary)
    m += pulp.lpSum(S * y[t] + h * I[t] for t in T)
    for t in T:
        prev = I[t - 1] if t > 0 else 0

```

```

        m += prev + x[t] - demand[t] == I[t]          # bilan de stock
        M_t = sum(demand[t:])                        # "big-M" le plus serré possible
        m += x[t] <= M_t * y[t]                      # pas de production sans lancement
    m.solve(pulp.PULP_CBC_CMD(msg=False))
    return [round(x[t].value()) for t in T], pulp.value(m.objective)

# =====
# PARTIE B – CLSP multi-produits avec capacité (MILP)
# =====
def clsp(allow_overtime=True, capacity=LINE_HOURS_PER_MONTH, msg=False, time_limit=60):
    K = list(PRODUCTS)
    d = {k: monthly_demand(k) for k in K}
    P = PRODUCTS
    m = pulp.LpProblem("CLSP", pulp.LpMinimize)
    x = pulp.LpVariable.dicts("prod", (K, T), lowBound=0, cat=pulp.LpInteger)
    I = pulp.LpVariable.dicts("stock", (K, T), lowBound=0)
    y = pulp.LpVariable.dicts("setup", (K, T), cat=pulp.LpBinary)
    O = pulp.LpVariable.dicts("overtime", T, lowBound=0,
                              upBound=OVERTIME_MAX if allow_overtime else 0)

    setup_c = pulp.lpSum(P[k]["setup_cost"] * y[k][t] for k in K for t in T)
    hold_c = pulp.lpSum(P[k]["hold_cost"] * I[k][t] for k in K for t in T)
    over_c = pulp.lpSum(OVERTIME_COST * O[t] for t in T)
    m += setup_c + hold_c + over_c

    for k in K:
        for t in T:
            prev = I[k][t - 1] if t > 0 else 0
            m += prev + x[k][t] - d[k][t] == I[k][t], f"bal_{k}_{t}"
            # big-M = min(demand restante, capacité max convertie en palettes)
            cap_units = (capacity + OVERTIME_MAX - P[k]["setup_time"]) / P[k]["unit_time"]
            M = min(sum(d[k][t:]), cap_units)
            m += x[k][t] <= M * y[k][t], f"setup_{k}_{t}"
    for t in T:
        m += (pulp.lpSum(P[k]["unit_time"] * x[k][t] + P[k]["setup_time"] * y[k][t]
                        for k in K) <= capacity + O[t]), f"cap_{t}"

```

```

t0 = time.perf_counter()
st = m.solve(pulp.PULP_CBC_CMD(msg=msg, timeLimit=time_limit))
dt = time.perf_counter() - t0
res = dict(status=pulp.LpStatus[st], time_s=dt)
if res["status"] != "Optimal":
    return res
res.update(total=pulp.value(m.objective), setup=pulp.value(setup_c),
           hold=pulp.value(hold_c), overtime=pulp.value(over_c))
res["prod"] = {k: [round(x[k][t].value(), 1) for t in T] for k in K}
res["stock"] = {k: [round(I[k][t].value(), 1) for t in T] for k in K}
res["setups"] = {k: [int(round(y[k][t].value())) for t in T] for k in K}
res["ot"] = [round(O[t].value(), 1) for t in T]
res["load"] = [sum(P[k]["unit_time"] * x[k][t].value() + P[k]["setup_time"] * y[k][t].value()
                  for k in K) for t in T]
res["n_vars"], res["n_cons"] = len(m.variables()), len(m.constraints)
return res

def independent_ww_plans():
    """Chaque produit planifie SEUL par Wagner-Whitin : ignore la capacite partagee."""
    out, load = {}, [0.0] * 12
    total = 0.0
    for k, P in PRODUCTS.items():
        d = monthly_demand(k)
        prod, cost = wagner_whitin(d, P["setup_cost"], P["hold_cost"])
        out[k] = prod
        total += cost
        for t in T:
            if prod[t] > 0:
                load[t] += P["unit_time"] * prod[t] + P["setup_time"]
    return out, load, total

if __name__ == "__main__":
    k = "A"
    P = PRODUCTS[k]
    d = monthly_demand(k)
    S, h = P["setup_cost"], P["hold_cost"]

    print("demande", d)
    for name, plan in [("LFL", lot_for_lot(d)), ("POQ", poq(d, S, h)[0]),
                      ("SM", silver_meal(d, S, h)), ("WW", wagner_whitin(d, S, h)[0]),
                      ("MILP", uncapacitated_milp(d, S, h)[0])]:
        c, n, _ = plan_cost(d, plan, S, h)
        print(f"{name:5s} {c:10.0f} setups={n} {plan}")
    ww, load, tot = independent_ww_plans()
    print("WW independants", round(tot), [round(l) for l in load])
    r = clsp()
    print(r["status"], round(r["total"]), round(r["setup"]), round(r["hold"]), round(r["overtime"]),
          round(r["time_s"], 2))
    print([round(l) for l in r["load"]], r["ot"])
    for kk in r["prod"]:
        print(kk, r["prod"][kk])
    r2 = clsp(allow_overtime=False)
    print("sans HS", r2["status"], r2.get("total"))

```

A.4 — level3_vrp.py

■ level3_vrp.py

"""

level3_vrp.py – NIVEAU OPERATIONNEL

Tournées de livraison depuis un entrepot (CVRP avec limite de distance)

- Formulation MILP a 2 indices + contraintes MTZ (charge et distance cumulees)
- Heuristiques : livraisons directes, plus proche voisin, Clarke & Wright (savings)
- Pre-traitement "camion complet" : les gros volumes partent en direct (FTL)

"""

import math

import time

import random

import pulp

 from data import (CLIENTS, DEPOTS, road_km, WORKING_DAYS, VRP_TRUCK_CAP,
 VRP_FIXED_PER_TRUCK, TRUCK_COST_KM)

WAVES_PER_DAY = 2 # 2 vagues de livraison JIT par jour (1 par poste)

L_MAX = 700 # km max par tournée (~9 h de conduite, reglement CE 561/2006)

Construction d'une instance : entrepot + clients affectes par le niveau 1

def build_instance(depot_name, client_names):

dep = [d for d in DEPOTS if d[0] == depot_name][0]

cl = [c for c in CLIENTS if c[0] in client_names]

nodes = [dep] + cl

q_full = [0] + [math.ceil(c[3] / (WORKING_DAYS * WAVES_PER_DAY)) for c in cl]

n = len(nodes)

dist = [[road_km(nodes[i], nodes[j]) if i != j else 0.0 for j in range(n)] for i in range(n)]

pre-traitement : camions complets en direct, reliquat dans les tournées

ftl = [q // VRP_TRUCK_CAP for q in q_full]

q = [qf % VRP_TRUCK_CAP for qf in q_full]

 return dict(names=[x[0] for x in nodes], coords=[(x[1], x[2]) for x in nodes],
 q=q, q_full=q_full, ftl=ftl, dist=dist, Q=VRP_TRUCK_CAP)

def route_len(route, dist):

path = [0] + route + [0]

```

    return sum(dist[path[k]][path[k + 1]] for k in range(len(path) - 1))

def solution_cost(routes, inst):
    km = sum(route_len(r, inst["dist"]) for r in routes)
    # camions complets directs (aller-retour)
    ftl_km = sum(2 * inst["dist"][0][i] * inst["ftl"][i] for i in range(len(inst["q"])))
    n_trucks = len(routes) + sum(inst["ftl"])
    cost = TRUCK_COST_KM * (km + ftl_km) + VRP_FIXED_PER_TRUCK * n_trucks
    return dict(cost=cost, km=km + ftl_km, trucks=n_trucks, routes=routes)

def feasible(route, inst):
    return (sum(inst["q"][i] for i in route) <= inst["Q"]
            and route_len(route, inst["dist"]) <= L_MAX + 1e-6)

# -----
# Heuristique 0 : un camion par client (reference "naive")
# -----
def direct_deliveries(inst):
    return solution_cost([i] for i in range(1, len(inst["q"])) if inst["q"][i] > 0, inst)

# -----
# Heuristique 1 : plus proche voisin (remplir le camion au plus pres)
# -----
def nearest_neighbor(inst):
    todo = {i for i in range(1, len(inst["q"])) if inst["q"][i] > 0}
    routes = []
    while todo:
        route, cur = [], 0
        while True:
            cands = [j for j in todo if feasible(route + [j], inst)]
            if not cands:
                break
            nxt = min(cands, key=lambda j: inst["dist"][cur][j])
            route.append(nxt); todo.remove(nxt); cur = nxt

```

```

        routes.append(route)
    return solution_cost(routes, inst)

# -----
# Heuristique 2 : Clarke & Wright (1964) – methode des economies
#   s_ij = d_0i + d_0j - d_ij : gain a servir i et j dans la meme tournée
# -----
def clarke_wright(inst):
    d = inst["dist"]
    custs = [i for i in range(1, len(inst["q"])) if inst["q"][i] > 0]
    routes = {i: [i] for i in custs} # au depart : un aller-retour par client
    where = {i: i for i in custs} # client -> identifiant de sa tournée
    savings = sorted(((d[0][i] + d[0][j] - d[i][j]), i, j)
                     for i in custs for j in custs if i < j), reverse=True)
    for s, i, j in savings:
        if s <= 0:
            break
        ri, rj = where[i], where[j]
        if ri == rj:
            continue
        A, B = routes[ri], routes[rj]
        # on ne fusionne que par les extremités des tournées
        merged = None
        if A[-1] == i and B[0] == j:
            merged = A + B
        elif A[0] == i and B[-1] == j:
            merged = B + A
        elif A[-1] == i and B[-1] == j:
            merged = A + B[:-1]
        elif A[0] == i and B[0] == j:
            merged = A[:-1] + B
        if merged and feasible(merged, inst):
            routes[ri] = merged
            del routes[rj]
            for c in merged:
                where[c] = ri
    return solution_cost(list(routes.values()), inst)

# -----
# Methode exacte : MILP a 2 indices, sous-tours elimines par MTZ
# -----
def vrp_milp(inst, time_limit=60, msg=False, warm_routes=None):
    d, Q = inst["dist"], inst["Q"]
    V = [0] + [i for i in range(1, len(inst["q"])) if inst["q"][i] > 0]
    C = V[1:]
    q = inst["q"]
    A = [(i, j) for i in V for j in V if i != j]

    m = pulp.LpProblem("CVRP", pulp.LpMinimize)
    x = pulp.LpVariable.dicts("x", A, cat=pulp.LpBinary) # arc (i,j) emprunte
    u = pulp.LpVariable.dicts("load", C, lowBound=0, upBound=Q) # charge livree cumulee
    D = pulp.LpVariable.dicts("km", C, lowBound=0, upBound=L_MAX) # km cumules a l'arrivee

    n_routes = pulp.lpSum(x[0, j] for j in C)
    m += (TRUCK_COST_KM * pulp.lpSum(d[i][j] * x[i, j] for (i, j) in A)
          + VRP_FIXED_PER_TRUCK * n_routes)

    for i in C:
        m += pulp.lpSum(x[i, j] for j in V if j != i) == 1 # on quitte chaque client 1 fois
        m += pulp.lpSum(x[j, i] for j in V if j != i) == 1 # on arrive chez chaque client 1 fois
    m += pulp.lpSum(x[0, j] for j in C) == pulp.lpSum(x[i, 0] for i in C)

    for i in C:
        m += u[i] >= q[i]
        m += D[i] >= d[0][i]
        # retour au depot dans la limite de distance
        m += D[i] + d[i][0] <= L_MAX + L_MAX * (1 - x[i, 0])
    for i in C:
        for j in C:
            if i != j:
                # MTZ charge : si j suit i, la charge cumulee augmente de q_j
                m += u[j] >= u[i] + q[j] - Q * (1 - x[i, j])
                # MTZ distance : si j suit i, la distance cumulee augmente de d_ij
                m += D[j] >= D[i] + d[i][j] - 2 * L_MAX * (1 - x[i, j])

```

```

# Demarrage a chaud (warm start) : on fournit une solution heuristique au solveur
if warm_routes:
    for a in A:
        x[a].setInitialValue(0)
    for r in warm_routes:
        path = [0] + r + [0]
        load, km = 0, 0.0
        for k in range(len(path) - 1):
            i, j = path[k], path[k + 1]
            x[i, j].setInitialValue(1)
            if j != 0:
                load += q[j]; km += d[i][j]
                u[j].setInitialValue(load); D[j].setInitialValue(km)

t0 = time.perf_counter()
log = f"/tmp/cbc_vrp_{random.randint(0, 10**9)}.log"
m.solve(pulp.PULP_CBC_CMD(msg=msg, timeLimit=time_limit, logPath=log,
                           warmStart=bool(warm_routes)))

dt = time.perf_counter() - t0
# PIEGE : m.status peut valoir "Optimal" meme quand la limite de temps a coupe
# la recherche. Le vrai statut de la solution est dans m.sol_status :
# 1 = optimum prouve ; 2 = solution entiere trouvee mais NON prouvee optimale
status = {1: "OPTIMAL", 2: "FEASIBLE"}.get(m.sol_status, "NO_SOLUTION")
bound = _read_cbc_bound(log)
if pulp.value(m.objective) is None:
    return dict(status="NO_SOLUTION", time_s=dt)
succ = {i: j for (i, j) in A if x[i, j].value() > 0.5}
routes = []
for j in C:
    if x[0, j].value() > 0.5:
        r, cur = [], j
        while cur != 0:
            r.append(cur); cur = succ[cur]
        routes.append(r)
res = solution_cost(routes, inst)
if status == "OPTIMAL" or bound is None:
    bound = res["cost"]

```

```

res.update(status=status, time_s=dt, n_vars=len(m.variables()), n_cons=len(m.constraints),
           bound=bound)
return res

def _read_cbc_bound(path):
    """Lit la meilleure borne inferieure dans le journal CBC (gap d'optimalite)."""
    try:
        txt = open(path).read()
    except OSError:
        return None
    import re
    found = re.findall(r"Lower bound:\s+([-\d.eE+]+)", txt)
    return float(found[-1]) if found else None

# -----
# Instances aleatoires pour le test de passage a l'echelle
# -----
def random_instance(n, seed, center=(49.45, 11.08), radius_deg=1.6):
    rng = random.Random(seed)
    dep = ("Depot", center[0], center[1])
    pts = [{"c%d" % k, center[0] + rng.uniform(-radius_deg, radius_deg) * 0.65,
            center[1] + rng.uniform(-radius_deg, radius_deg)} for k in range(n)]
    nodes = [dep] + pts
    dist = [[road_km(nodes[i], nodes[j]) if i != j else 0.0 for j in range(n + 1)]
            for i in range(n + 1)]
    q = [0] + [rng.randint(3, 12) for _ in range(n)]
    return dict(names=[p[0] for p in nodes], coords=[(p[1], p[2]) for p in nodes],
               q=q, q_full=q, ftl=[0] * (n + 1), dist=dist, Q=VRP_TRUCK_CAP)

if __name__ == "__main__":
    from level1_cflp import solve_cflp
    net = solve_cflp()
    for dep in net["open"]:
        cl = [k for k, v in net["assign"].items() if dep in v]
        inst = build_instance(dep, cl)

        print(dep, inst["q"], inst["ftl"])
        for name, f in [("direct", direct_deliveries), ("NN", nearest_neighbor),
                        ("CW", clarke_wright), ("MILP", vrp_milp)]:
            r = f(inst)
            print(f" {name:6s} {r['cost']:8.0f} EUR {r['km']:6.0f} km trucks={r['trucks']}"
                  f" {[inst['names'][i].split(' ')[0] for i in rt] for rt in r['routes']}"
                  + (f" t={r['time_s']:.2f}s {r['status']}" if 'time_s' in r else ""))

```

A.5 — run_all.py

■ run_all.py

```

"""
run_all.py – Rejoue TOUTES les experiences du guide et sauvegarde results.json
Usage : python run_all.py          (~6-8 minutes, dont ~5 min de test d'echelle VRP)
        python run_all.py --fast    (saute le test d'echelle)
"""

import json
import sys
import statistics
import time

import data as D
from level1_cflp import solve_cflp, kmeans_heuristic, evaluate_fixed_network
import level2_lotsizing as L2
import level3_vrp as V

R = {}
t_start = time.perf_counter()

# ===== NIVEAU 1
print("== Niveau 1 : localisation d'entrepots")
base = solve_cflp()
R["l1_base"] = base
R["l1_multi"] = solve_cflp(single_sourcing=False)
R["l1_lp_strong"] = solve_cflp(relax=True, strong=True)
R["l1_lp_weak"] = solve_cflp(relax=True, strong=False)
R["l1_weak_milp"] = solve_cflp(strong=False) # meme optimum, autre temps de calcul

R["l1_p_sweep"] = []
for p in range(1, 8):
    r = solve_cflp(p=p)
    R["l1_p_sweep"].append(dict(p=p, status=r["status"], total=r.get("total"),
                                fixed=r.get("fixed"), inbound=r.get("inbound"),
                                handling=r.get("handling"), outbound=r.get("outbound"),
                                open=r.get("open")))

fm_grid, om_grid = [0.5, 0.75, 1.0, 1.5, 2.0], [0.6, 1.0, 1.5, 2.0, 2.5]
R["l1_robust"] = dict(fixed=fm_grid, out=om_grid, cells=[])
for fm in fm_grid:

```

```

row = []
for om in om_grid:
    r = solve_cflp(fixed_mult=fm, out_mult=om)
    row.append(dict(open=r["open"], total=r["total"]))
R["l1_robust"]["cells"].append(row)

km = []
for seed in range(20):
    cents, chosen = kmeans_heuristic(3, seed)
    st, val = evaluate_fixed_network(chosen)
    km.append(dict(seed=seed, open=chosen, status=st, total=val if st == "Optimal" else None,
                  centroids=cents))
R["l1_kmeans"] = km
feas = [k["total"] for k in km if k["total"]]
R["l1_kmeans_summary"] = dict(n=len(km), n_feasible=len(feas), best=min(feas), worst=max(feas),
                              mean=statistics.mean(feas))
R["l1_port_hamburg"] = solve_cflp(port=("Hamburg (port)", 53.54, 9.97))

# ===== NIVEAU 2
print("== Niveau 2 : lot-sizing")
R["l2_demand"] = {k: D.monthly_demand(k) for k in D.PRODUCTS}
R["l2_single"] = {}
for k, P in D.PRODUCTS.items():
    d, S, h = D.monthly_demand(k), P["setup_cost"], P["hold_cost"]
    plans = {"Lot-for-Lot": L2.lot_for_lot(d),
            "POQ (Wilson)": L2.poq(d, S, h)[0],
            "Silver-Meal": L2.silver_meal(d, S, h),
            "Wagner-Whitin": L2.wagner_whitin(d, S, h)[0]}
    t0 = time.perf_counter()
    milp_plan, milp_val = L2.uncapacitated_milp(d, S, h)
    plans["MILP (CBC)"] = milp_plan
    out = {}
    for name, plan in plans.items():
        c, n, stock = L2.plan_cost(d, plan, S, h)
        out[name] = dict(cost=c, setups=n, plan=plan, stock=stock)
    out["_eoq"] = L2.poq(d, S, h)[1]
    R["l2_single"][k] = out

```

```

ww, load, ww_total = L2.independent_ww_plans()
R["l2_ww_indep"] = dict(plans=ww, load=load, total=ww_total)
R["l2_clsp"] = L2.clsp()
R["l2_clsp_no_ot"] = L2.clsp(allow_overtime=False)
R["l2_cap_sweep"] = []
for cap in [600, 615, 630, 645, 660, 690, 720]:
    r = L2.clsp(capacity=cap)
    R["l2_cap_sweep"].append(dict(cap=cap, status=r["status"], total=r.get("total"),
                                setup=r.get("setup"), hold=r.get("hold"),
                                overtime=r.get("overtime"),
                                ot_hours=sum(r["ot"]) if "ot" in r else None,
                                n_setups=sum(sum(v) for v in r["setups"].values()) if "setups" in r
                                else None))

# ===== NIVEAU 3
print("== Niveau 3 : tournées")
V.WAVES_PER_DAY = 2
R["l3"] = {}
for dep in base["open"]:
    cl = [c for c, v in base["assign"].items() if dep in v]
    inst = V.build_instance(dep, cl)
    res = {}
    for name, f in [("Livraisons directes", V.direct_deliveries),
                  ("Plus proche voisin", V.nearest_neighbor),
                  ("Clarke & Wright", V.clarke_wright),
                  ("MILP (CBC)", V.vrp_milp)]:
        r = f(inst)
        r["routes_named"] = [[inst["names"][i] for i in rt] for rt in r["routes"]]
        res[name] = r
    res["_inst"] = dict(names=inst["names"], coords=inst["coords"], q=inst["q"])
    R["l3"][dep] = res

# ===== INTEGRATION
print("== Integration strategique <-> operationnel")
Dmap = {d[0]: d for d in D.DEPOTS}

def vrp_annual(net, waves):

```

```

V.WAVES_PER_DAY = waves
tot, palkm, det = 0.0, 0.0, {}
for dep in net["open"]:
    cl = [c for c, v in net["assign"].items() if dep in v]
    inst = V.build_instance(dep, cl)
    r = V.vrp_milp(inst, time_limit=60)
    det[dep] = dict(cost_day=r["cost"], trucks=r["trucks"], status=r["status"],
                    n_clients=len(cl), routes=[[inst["names"][i] for i in rt] for rt in
r["routes"]])
    tot += r["cost"] * waves * D.WORKING_DAYS
for c in D.CLIENTS:
    dep = max(net["assign"][c[0]], key=net["assign"][c[0]].get)
    palkm += c[3] * D.road_km(Dmap[dep], c)
return tot, tot / palkm, det

R["int_waves"] = {}
for w in (1, 2):
    tot, rate, _ = vrp_annual(base, w)
    R["int_waves"][w] = dict(annual=tot, rate=rate)

loop, mult = [], 1.0
for it in range(5):
    net = solve_cflp(out_mult=mult)
    tot, rate, det = vrp_annual(net, 2)
    true_total = net["fixed"] + net["inbound"] + net["handling"] + tot
    loop.append(dict(it=it, mult=mult, open=net["open"], est_total=net["total"],
                    est_outbound=net["outbound"], vrp_outbound=tot, true_total=true_total,
                    rate=rate, fixed=net["fixed"], inbound=net["inbound"],
                    handling=net["handling"], assign=net["assign"], det=det))
    new_mult = rate / D.OUTBOUND_EUR_PAL_KM
    if abs(new_mult - mult) < 0.02:
        break
    mult = new_mult
R["int_loop"] = loop

# ===== ECHELLE VRP
if "--fast" not in sys.argv:

```

```

print("== Test de passage a l'echelle VRP (peut prendre ~5 min)")
V.WAVES_PER_DAY = 2
R["l3_scale"] = []
for n in [8, 10, 12, 15, 20, 25]:
    inst = V.random_instance(n, seed=1)
    cw = V.clarke_wright(inst)
    cold = V.vrp_milp(inst, time_limit=30)
    warm = V.vrp_milp(inst, time_limit=30, warm_routes=cw["routes"])
    R["l3_scale"].append(dict(n=n, cw=cw["cost"], cold=cold["cost"], cold_status=cold["status"],
                             cold_bound=cold["bound"], cold_time=cold["time_s"],
                             warm=warm["cost"], warm_status=warm["status"],
                             warm_bound=warm["bound"], warm_time=warm["time_s"],
                             n_vars=cold["n_vars"], n_cons=cold["n_cons"]))
    print(" n=%d CW=%.0f cold=%.0f (%s) warm=%.0f (%s)" % (
        n, cw["cost"], cold["cost"], cold["status"], warm["cost"], warm["status"]))

R["runtime_s"] = time.perf_counter() - t_start
json.dump(R, open("results.json", "w"), indent=1, default=str)
print("Termine en %.0f s -> results.json" % R["runtime_s"])

```

A.6 — scale_test.py

■ scale_test.py

```

"""scale_test.py – test de passage a l'echelle du VRP (ajoute a results.json)
Usage : python scale_test.py 8 10 12 15 20 25"""
import json, sys
import level3_vrp as V
V.WAVES_PER_DAY = 2
R = json.load(open("results.json"))
R.setdefault("l3_scale", [])
for n in map(int, sys.argv[1:]):
    inst = V.random_instance(n, seed=1)
    cw = V.clarke_wright(inst)
    cold = V.vrp_milp(inst, time_limit=30)
    warm = V.vrp_milp(inst, time_limit=30, warm_routes=cw["routes"])
    R["l3_scale"] = [s for s in R["l3_scale"] if s["n"] != n]
    R["l3_scale"].append(dict(n=n, cw=cw["cost"], cold=cold["cost"], cold_status=cold["status"],
                             cold_bound=cold["bound"], cold_time=cold["time_s"],
                             warm=warm["cost"], warm_status=warm["status"],
                             warm_bound=warm["bound"], warm_time=warm["time_s"],
                             n_vars=cold["n_vars"], n_cons=cold["n_cons"]))
    R["l3_scale"].sort(key=lambda s: s["n"])
    json.dump(R, open("results.json", "w"), indent=1, default=str)
    print("n=%d CW=%.0f cold=%.0f %s (%.1fs, LB %.0f) warm=%.0f %s (%.1fs, LB %.0f)" % (
        n, cw["cost"], cold["cost"], cold["status"], cold["time_s"], cold["bound"],
        warm["cost"], warm["status"], warm["time_s"], warm["bound"]), flush=True)

```

B Annexe — glossaire

Français · English · Deutsch

Terme	Définition
Big-M	Constante qui active/désactive une contrainte via une binaire. À choisir la plus petite possible.
Branch-and-bound	Algorithme des solveurs MILP : séparer sur les variables fractionnaires, élaguer grâce aux bornes.
CFLP	Capacitated Facility Location Problem — localisation d'entrepôts avec capacités.
CLSP	Capacitated Lot-Sizing Problem — lot-sizing multi-produits sur ressource partagée.
CVRP	Capacitated Vehicle Routing Problem — tournées avec capacité des véhicules.
Gap d'optimalité	(solution – borne) / solution. 0 % = optimum prouvé.
Heuristique	Méthode rapide sans garantie d'optimalité (Silver-Meal, Clarke & Wright...).
MTZ	Miller-Tucker-Zemlin : contraintes d'ordre qui éliminent les sous-tours.
Relaxation LP	Le MILP où les variables entières deviennent continues ; fournit une borne.
Single sourcing	Chaque client est servi par un seul entrepôt.
Warm start	Solution de départ fournie au solveur pour accélérer la recherche.
Wagner-Whitin	Algorithme de programmation dynamique optimal pour le lot-sizing sans capacité.